(VIDEO 1)



Perfil del Licenciado en sistemas:

- Enfoque hacia la resolución de problemas
- Es observador
- Visión global del problema en su contexto
- Análisis y diseño modular de soluciones → separar y tratar de forma individual cada cosa
- Cultura de la simpleza

¿QUÉ ES UN SOFTWARE? Solución informática a una necesidad de negocio (problema). Se compone de programas, datos y documentación. Se debe documentar el desarrollo y el mantenimiento. Es un desarrollo lógico y a medida (los sistemas se hacen una única vez pero se van adaptando según las necesidades específicas) **que no tiene repuestos para su mantenimiento ya que se deteriora.**
Para la construcción de un software de **calidad** necesito **buenas prácticas** (estándar de proceso de desarrollo) y **disciplina**.
De un software se espera:

- Que sea fiable → que haga aquello para lo que fue construido
- Que sea eficiente → realiza solo aquello que es necesario (recursos informáticos, tiempos de respuesta)
- Que sea amigable → intuitivo y facilitador con poca capacitación
- Que sea usable → que logre la inmediata adaptación de la tarea a lo que el software exige
- Que sea fácil de mantener (+ bien documentado)
- Que las entregas sean frecuentes → agilidad y manejo de la ansiedad del usuario

Tipos de software: procesamiento de información/transacciones/registro, de oficina, de trabajo de conocimiento, administrativos, soporte de decisiones, soporte de ejecutivos, inteligencia artificial/sistemas expertos.

Vistas de la calidad del software → Actualmente se evalúa la calidad con 3 vistas:

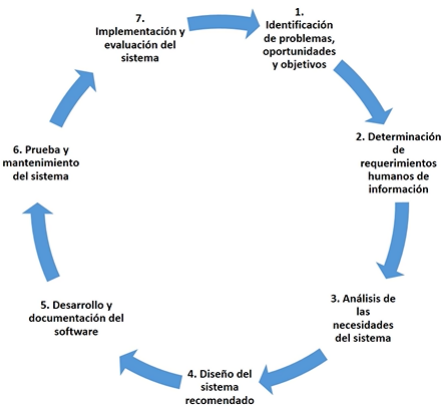
- ➔ **Producto (visión antigua)**: Prioriza que la aplicación cumpla con sus funciones esenciales. Comparación con una matriz de construcción para garantizar que el producto final esté correctamente elaborado. Como el producto da respuesta a los requerimientos del usuario. Los requerimientos puede ser:
  - ◆ Normales: identificados y expuestos en documentación formal
  - ◆ Esperados: aquellos que no están escritos pero que son lógicos que aparezcan. Ej: que sea ordenado o no usar colores de river si trabajo con boca.
  - ◆ Emocionantes: una vez que el sistema está hecho, el usuario los identifica, le genera nuevas necesidades.
- ➔ **Proceso**: actividades estandarizadas de acuerdo al resultado → el producto va a ser de calidad si el proceso es de calidad. Énfasis en procesos estandarizados y homogeneización. Prioridad en garantizar que todos los productos sean tratados de la misma manera. Paralelo con los principios del toyotismo, buscando eficiencia y consistencia en la producción.
- ➔ **Usuario (visión actual)**: Enfoque en el producto, proceso y usuario. Se recibe la calidad con la percepción del usuario. Consideración de la calidad en todas las etapas del proceso. Atención a la satisfacción del usuario y a la experiencia de uso en el momento de verdad de la aplicación. Enfoque en la mejora continua en el desarrollo y producción de software.

ISO 25000 → estándar de calidad en los softwares.
Principios para el desarrollo de aplicaciones informáticas:

DAR VALOR AL USUARIO	**DETERMINAR EL PROBLEMA LO ANTES POSIBLE**
PENSAR	SER SIMPLES Y CLAROS
OTROS CONSUMEN LO QUE SE PRODUCE	**MANTENER LA VISIÓN (COMPARTIDA)**
ENTREGAR LO ANTES POSIBLE	PRIMERO LO CORRECTO LUEGO LO RÁPIDO
PERSONAS Y TIEMPO NO SE INTERCAMBIAN	**LA GENTE ES CLAVE DEL ÉXITO**
ASUMIR RESPONSABILIDAD	COMPROMISO CON EL CLIENTE
ABRAZAR AL FUTURO	**PLANEAR LA REUTILIZACIÓN**
BUENAS PRÁCTICAS	MEJORA CONTINUA

(VIDEO 2)

CICLO DE VIDA DEL SOFTWARE (enfoque metodológico → agrupa actividades y establece instancias):



1. **Identificación de problemas, oportunidades y objetivos:** a través de la visión global se debe poder conceptualizar el problema, en términos de necesidad y de solución esperada. Se debe indagar el entendimiento de la necesidad (colaborar a que el cliente tenga en claro que es lo que necesita), el objetivo (afirmación de lo que se quiere de la aplicación, norte/éxito del proyecto) y el alcance (límites del aplicativo). Se establecen oportunidades, restricciones (de tiempo, presupuesto, recursos) y volatilidad (cambios en los requerimientos a lo largo del proyecto).
2. **Determinación de requerimientos humanos de información:** se basa en la interacción humana-computadora para definir necesidades del usuario y escenarios de usuarios. Se busca comprender cómo los usuarios realizan su trabajo en la actualidad con sistemas computarizados y la utilidad del nuevo sistema en el trabajo.
3. **Análisis de las necesidades del sistema:** se determinan concretamente los requerimientos → modelado (diagrama de flujo, de clases, etc.) → el modelado tiene en cuenta: funcionalidades, datos, escenarios de uso, características y restricciones. En esta instancia se debe negociar, especificar, validar y administrar los cambios entre cliente y desarrollador.
    - **Tips para la negociación**: interés de los de distintos actores, escucha activa, win-win, pactar → formalizar, identificar tipos de negociadores (visceral, activista, competitivo, colaborador)
4. **Diseño del sistema recomendado:** diseño que tiene lógica y llega a definir el sistema informático a entregar. Arquitectura + Interfaz (como la aplicación se comunica con su entorno) + Procedimientos (reglas de negocio y controles). Permite administrar y gestionar el proyecto.
5. **Desarrollo y documentación del software:** programación, manual de usuario y de procedimientos, documentación, capacitación y ayudas.
6. **Prueba y mantenimiento del sistema:** no se puede entregar sin probar pero tampoco se puede probar todo, se busca el error.
   Probar antes de entregar → estrategia, planificar, programadores/desarrolladores/testers/usuarios.
   Mantener todo el tiempo → corrección, mejora, actualización y reestructuración.
7. **Implementación y evaluación del sistema:** se tiene que facilitar la implementación, planificarla y capacitar. Se debe evaluar al software puesto en producción respecto de la percepción del usuario en cuanto a su calidad.

(VIDEO 3)

METODOLOGÍAS DE DESARROLLO DE SOFTWARE



Los paradigmas definen el enfoque que se le va a dar a la solución informática. Los lenguajes toman esos enfoques para dar las herramientas que permitan realizar la aplicación desde ese paradigma.
Todos los lenguajes están basados en uno o más paradigmas (multiparadigmas).

Lenguajes - Paradigmas:

- **Programación orientada a procedimientos:** programa construido a partir de sentencias que hacen a un procedimiento. El mismo programa es quien hace el control, por lo que son conductistas. Ej: Pascal, C#
- **Programación orientada a eventos:** El usuario administra el evento, le dice a la aplicación que es lo que tiene que hacer. Ej: Visual Basic.
- **Programación funcional:** orientado a funciones matemáticas y cálculos. No define variable a priori, sino que las define el usuario.
- **Programación lógica:** axiomas y reglas que relacionan objetos/datos. Tiene instancias predicativas que facilitan la toma de decisiones.
- **Programación orientada a objetos**: solución del problema por la descomposición del mismo con objetos que interactúan. Ej: Java, CSharp
- **Programación orientada a aspectos:** establece módulos de trabajos.

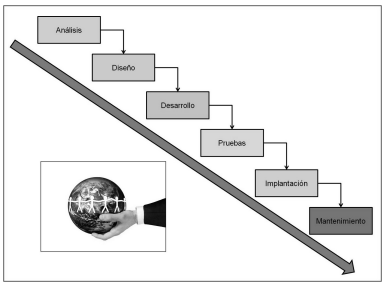
- **Múltiples paradigmas**

Modelo general de proceso:



Estándar de trabajo → forma en la que se define el proceso. El proceso contempla un conjunto de actividades y a su vez esas actividades tienen tareas: comunicación, planeación, modelado, construcción y despliegue.

Metodologías de desarrollo:



- **Cascada (Tradicional)**: Hace todo de una, entrega un producto llave en mano. Actualmente no existe ya que se debería tener bien definido desde el principio los requerimientos y los riesgos conocidos (+ manejo de la ansiedad del cliente/usuario).
    - Lineal secuencial → una detrás de otra y hay una única entrega final
    - Ciclo de vida clásico
    - Requerimientos conocidos y reducidos
    - Riesgo conocido
    - No debería haber cambios (desventaja)
- **Prototipos**: Es una muestra de lo que se va a entregar sin datos reales. Está bueno demostrar una prueba por cada requerimiento. No están en producción. No es un modelo prescriptivo. No son modelos integrales de desarrollo.
    - Recopilación de información
    - Refinamiento de requerimientos
    - Reaccion de usuarios
    - Reducción de riesgos de requerimientos
    - Ansiedad del usuario (desventaja)
- **Desarrollo rápido de aplicaciones**: Hacer casi en paralelo varios en cascada, tratando de intercalarlos de forma tal de maximizar los recursos. Varios grupos trabajando en simultáneo.
    - Lineal secuencial paralelo (cascada en paralelo)
    - Division funcional y equipos (grupos ad-hoc)
    - Lapsos de tiempo menores a cascada
    - Cooperación con el cliente
    - Requiere de muchos recursos en simultáneo (desventaja)
- **Incremental**: Construcción paulatina de etapas de un producto, ir haciendo entregas parciales, siendo fundamental un buen diseño que soporte eso. La gran mayoría de los requerimientos no son fijos ni determinados desde el principio. Funciona en la actualidad. Los entregables parciales generan un costo adicional.
    - Funciones-Iteraciones → agregar valor en cada iteración
    - Secuencial lineal con iteraciones
    - Sin definición inicial → no se invierte todo desde el inicio.
    - Evolución en el tiempo de los requerimientos
    - Definición de prioridades
- **Espiral:** Tienen grandes centros de cómputos, siempre se le agrega algo al sistema. Hay un requerimiento genérico pero con extras ya que no se tienen todos los recursos desde el principio (mini proyectos). El sistema se termina de desarrollar cuando hay que implementarlo. Estilos constructivos basados en datos. Se termina de hacer el sistema cuando se desea cambiar de sistema.
    - Cascada + Iterativo
    - Miniproyectos
    - Control del riesgo
    - Administración minuciosa

    - Estructura exclusiva
- **Metodologías Ágiles:** Entrega rápida y desburocratización del proceso de desarrollo. Se documenta únicamente lo necesario. Se entiende rápidamente que es lo que se necesita y requiere de feedback constante. Hay una discusión permanente de recursos. Se prueba desde el momento 0. Contraejemplo: No sirve para implementar sistemas bancarios dado que tienen muchisimos requerimientos de documentación. Recomendaciones: Diseño robusto, Comunicación fluida, Construcción simple, Retroalimentación permanente dentro del grupo, Toma de decisión siempre, Proceso bien internalizado.
    - Iterativo - Evolutivo
    - Flexibilidad del proceso (requerimientos cambiantes)
    - Entrega rápida
    - Valor al negocio
    - Mínima ingeniería
    - Sencillez
    - Trabajo colaborativo
    - Equipo ágil
    - Asignación de presupuesto y tiempos
    - Principios
- **Programación extrema:**
    - Es un tipo de metodología ágil
    - Comunicación
    - Simpleza
    - Retroalimentación
    - Toma de decisiones
    - Buenas prácticas
    - Procesos

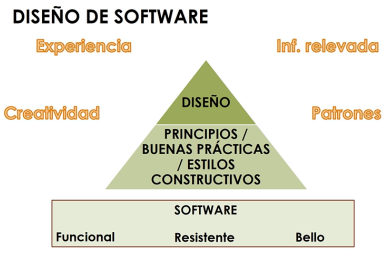
(VIDEO 4)

FUNDAMENTOS DEL DISEÑO DE SOFTWARE

El diseño se basa en la creatividad del codificador. Es importante rastrear desde el requerimiento hasta su construcción. Es importante lograr arquitectura de diferentes niveles (funcionales, clase, interfaz)
1° Se piensa en la salida, 2° entrada y 3° procesos.
Primero se analiza un problema para luego diseñar una solución

- ➔ El problema se trabaja primero desde un nivel alto de abstracción y luego desde uno bajo.
- ➔ Luego, esto se modela a través de una de las metodologías y de esa forma se determina las necesidades de usuario y de negocio y las restricciones que podría haber (financieras, tecnológicas, etc)

En la instancia de diseño se busca darle una solución a la problemática que teníamos. *Definimos un plano/esqueleto de la solución que va a servir como guía para la administración del proyecto, definiéndonos los entregables y además es un elemento de comunicación de las decisiones que tomamos a la hora de construir.*
Para el diseño de software se necesita de:



- Experiencia
- Creatividad
- Patrones (situaciones exitosas del pasado que reutilizamos)
- Información revelada (requerimientos y restricciones)

Todo esto tiene que estar soportado en principios, buenas prácticas y estilos constructivos.
El software tiene que ser: (perspectiva del usuario)

- Funcional
- Resistente/escalable
- Bello
- Óptimo
- Entendible

Principios del diseño:

- **Rastreabilidad**: Trazabilidad entre el diseño y los requisitos
- **Arquitectura**: Panorama general y completo de la aplicación.
    - Nos va a dar una integridad conceptual de lo que es la aplicación

    - Nos va a permitir enfocar todas las acciones
    - Vamos a poder ver mejor el alcance de la solución
- **Datos/Funciones**: Considerar tanto los datos que va a necesitar la solución como las funciones.
- **Iteración**: Refinamiento del diseño (desde el alto nivel al bajo).
- **Facilidad**: El modelado tiene que ser simple para que los que construyan la aplicación puedan entenderlo sin problemas. Es importante acordar con los usuarios sobre las necesidades que tienen.
- **Interfaz**: Tiene que ser fácil de manejar para que ante cualquier error poder solucionarlo fácilmente
- **Interfaz de usuario**: Hay que tener en cuenta quien va usar la aplicación, cuáles son las características del usuario y en qué contexto, para diseñar una solución de acuerdo a sus necesidades.
- **Componentes**: Diseñar el problema en muchos componentes y que entre ellos sean independientes, el acoplamiento sea mínimo (para que no se complique el mantenimiento y las pruebas). Permiten trabajar en la administración de los entregables, logrando una implementación evolutiva/incremental.

Buenas prácticas de diseño:

- ➔ **Niveles de abstracción:** Poder llegar al nivel más bajo de abstracción para poder lograr una mejor construcción de la solución. Vamos a tener distintas vistas: Desde la lógica (problema, vista de usuario), desde la posibilidad de llevar adelante el proyecto, desde la implementación del proceso, desde la implementación física.
- ➔ **Modularización:** Poder abordar los problemas por separado, trabajar en módulos (ventas, tesorería, RRHH, etc). Hay que tener en cuenta:
    - ◆ El costo (mientras más descompongo más costoso va a ser integrar luego), hay que buscar un punto de equilibrio, ni modularización excesiva ni escasa. Si es excesiva problema de errores y mucha interacción, pérdida de recursos y no descomponer afecta la lógica del módulo.
    - ◆ La cohesión del módulo, el módulo solo realiza su función, no realiza otra cosa.
    - ◆ El ocultamiento de información (técnica o transaccional): Los otros módulos no deben poder acceder a la información de otro módulo si no es necesario. Lo mismo para los usuarios, no deben poder acceder a información que no necesitan.
- ➔ **Patrones:** Reutilizar componentes que diseñamos anteriormente para disminuir costos y tiempos.
- ➔ **Modelos-Refinamiento:** Usar modelos de alto a bajo nivel de abstracción con refinamiento para que sea simple documentar y comunicar.
- ➔ **Arquitectura de negocio:** Es importante entender primero las áreas del negocio, luego los procesos de esas áreas y finalmente las funciones de esos procesos de negocio. Estas funciones nos van a permitir identificar los distintos módulos. Una función puede pertenecer a más de un proceso. Es importante saber cuáles son los procesos de apoyo, los procesos primarios y los de gestión para establecer la priorización de desarrollo.

Criterios de buen diseño: Que no sea complejo, que tenga solo lo necesario, que se pueda reutilizar, que sea visible, fácil de entender, económico, etc.

Modelo de organización: Nos dice como la entidad se conceptualiza para enfrentar el negocio.
Todo esto nos va a permitir confeccionar la **arquitectura de negocio.**
Modelos de la arquitectura de negocio:

- **Lógico:** Nos permiten tener la arquitectura funcional.
- **Clases/Datos**
- **Usuario:** Interfaces de usuario, manipulación de entradas y salidas de datos.
- **Técnico de desarrollo:** Módulo hacia adentro (programas). Maneja la lógica más dura del negocio.

Con esto vamos a poder mapear el negocio, la organización y la aplicación a desarrollar → mapeo = diseño.

Modelos de análisis de diseño:

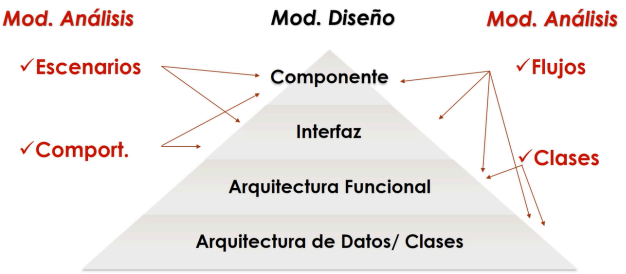
- **De escenario:** Podemos trabajar como input para definir el diseño a nivel de interfaz o componentes (niveles bajos). Ej: diagramas de carril, de canal.
- **De comportamiento:** Trabajar en el nivel bajo de interfaz o componentes. Ej. estado de secuencia, diagrama de clase.

- **De flujo:** Se trabaja tanto en los niveles altos (arquitectura funcional o de datos/clases) o bajos.
- **De clases:** Se trabaja en los niveles altos (arquitectura funcional o de datos/clases).

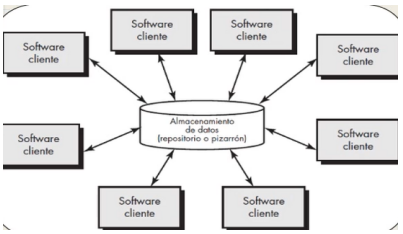
Dependiendo del nivel de abstracción con el que trabajamos elegiremos alguno de estos modelos.

Niveles en el diseño: (del más alto al más bajo)

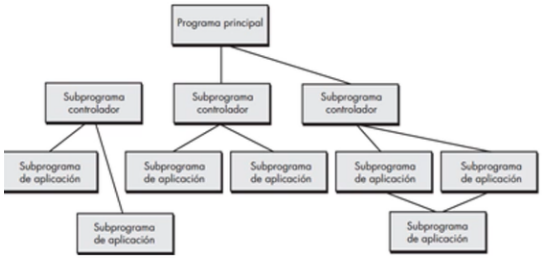
- **Arquitectura de datos/clases:** Base del esquema. Pasa del análisis al diseño de las clases. Se trabaja con diagramas de entidad-relación → permite identificar los datos que vamos a necesitar para la solución. Se define el marco conceptual.
- **Arquitectura funcional:** Define el alcance de la aplicación a partir del conjunto de módulos, la coexistencia, coordinación y la interconexión de los módulos. Define el plano general de la aplicación, como las funcionalidades van a interactuar con los datos y clases. Se trabaja con diagramas de contexto (identifica sistemas, controladores, usuarios, dispositivos, etc).
- **Interfaz:** Condicionan los componentes que se van a usar y cómo van a actuar. De usuario y de aplicaciones
- **Componentes**

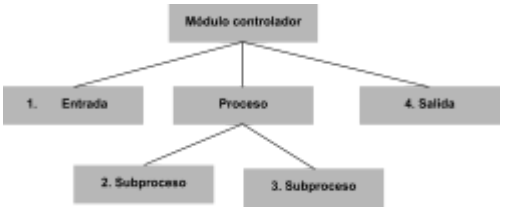


Tipos de diseños/estilos: (combinables entre sí)



- **Centrado en datos:** una base de datos + funcionalidades. La base de datos tiene las reglas del negocio. Óptimo para gran modelo de datos. Los módulos solo se relacionan a través de la base de datos. Típico en sistemas de módulos y en módulos de espiral.
- **Retorno/Organigrama:** Sube y baja de niveles llevando y trayendo los datos por estos niveles. Doy el control de la operación y retorna. Es un modelo jerárquico, hay programas principales (que invocan y controlan) y subprogramas (que gestionan lo invocado y devuelven el control al sistema superior). *Todos los procesos tienen entrada y salida.* Es un estilo fácil de lograr a partir de los DFD. Puede ser un problema para aplicaciones muy grandes.

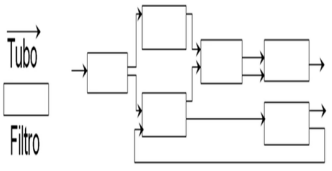




- **Capas/En espiral:** manejan elementos propios de las reglas de negocio (de comportamiento/de gestión) en cada uno de los niveles. (interfaz de usuario, programas, e/s, procesos, memoria, hardware, etc). Dentro de cada uno puede haber otros estilos. Cada vez que se cambia algo en un nivel se tiene que analizar en qué medida impacta hacia adentro y hacia afuera. Se usa mucho en programación orientada a objetos.



- **Flowchart (Modelo de flujo)**: Define instancias de transacciones. Hago módulos como instancias de una transacción. Trabaja con entradas y luego las procesa para lograr salidas. Se usa mucho en modelos de transacciones que exigen transformaciones y autorizaciones (ej: seguimiento de expedientes).



(VIDEO 5)

ANÁLISIS Y DISEÑO ESTRUCTURADO DE SOFTWARE

Análisis y Diseño → etapas estancas que se diferencian más las etapas: El análisis es mucho más cercano al negocio y el diseño es más cercano al software.

↳ Identifican y organizan los módulos de trabajo con el uso de herramientas: en el análisis se usan los DFD (diagramas de flujo de datos) y en el diseño se usan diccionarios de datos, diagramas de entidad-relacion, árbol de invocación, diagrama de entrada-proceso-salida, etc.

Análisis estructurado de Software:

Diagrama de flujo de datos (DFD): Herramienta para el análisis estructurado de software. Permite ver el movimiento de los datos de la organización y cómo se van transformando, tanto para dentro como fuera de la organización.

Ventajas:

- ➔ Permite ver a la organización y al SW sin una implicancia técnica. Vemos como la organización toma el negocio y como el SW va a tomar lo que hace la organización
- ➔ No visualiza inicialmente aspectos físicos
- ➔ Conceptualiza (el negocio dentro de la aplicación informática)
- ➔ Tiene una comprensión de relación de niveles de sistemas
- ➔ Fácil comunicación, tanto hacia adentro como hacia afuera (el usuario)
- ➔ Permite presentarle un sistema propuesto al usuario
- ➔ No identifica procesos manuales y automáticos

Elementos del diagrama:

- ➔ **Entidades ( ):** Entes externos a la aplicación que me generan impulso esperando o no respuesta para que la aplicación haga algo o también pueden ser receptores de información de la aplicación. Son sustantivos (personas, áreas, un tercero, otra aplicación, etc).
- ➔ **Flujos (→):** Representan como ingresan o egresan los datos en la aplicación. Son sustantivos compuestos (datos: orden de pedido, cheques al pago).
- ➔ **Almacenes**:  Buscan identificar los datos de negocio (manuales o automáticos) persistentes. No se enfoca en los datos temporales. Suelen repetirse en varios lugares del diagrama (por esto se usa un carácter numérico al principio para diferenciarlos). Son sustantivos (asociados al nombre del dato que almacena). Por ejemplo, datos de clientes. No datos transaccionales.
- ➔ **Procesos (◯)**: Tres niveles (sistemas, subsistemas y los procesos). Estos últimos serán los que quedarán a la hora de la construcción del sistema. Tiene un verbo (la acción que se realiza), un sustantivo (asociado al resultado del proceso y un adjetivo (especifica mejor la salida).

Reglas:

- Por lo menos tiene que haber un proceso. **Si hay un solo proceso es un diagrama de contexto. El proceso es el gran sistema.**
- No puede haber objetos independientes
- **Los procesos tienen que tener si o si flujos de entradas y de salidas.**
- **Los almacenes solo se conectan con procesos**, NO con entidades ni entre sí.

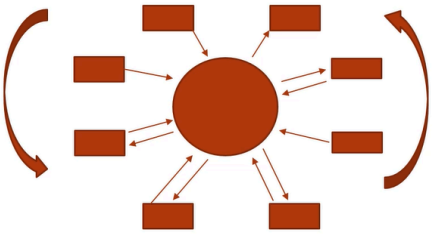
Pasos:

1. **Crear un diagrama de contexto** → Muestra la aplicación en su ámbito de desenvolvimiento.
2. **Crear un diagrama nivel 0** (a nivel de subsistemas)
3. **Crear diagramas hijo según necesidad** → Van a estar los procesos en sus funciones específicas
4. **Crear diagramas físicos** → Muestran las actividades en su contexto físico
5. **Identificar flujos de transformación y flujos de transacción**
6. **Diseño de alto nivel**

DIAGRAMA DE CONTEXTO

- Es la primera visión del sistema
- El sistema informático en su contexto → macro proceso: sistema como unidad.
- Permite ver el sistema en el contexto en el que funcionará y hacer una separación con la aplicación

- Define el alcance y los límites → Todo lo que está en el diagrama es lo esperable del sistema, es el ámbito en donde el sistema va a trabajar y cómo lo hará.
- El sistema se va a vincular con el contexto a través de entidades o terminales que le van a exigir al sistema para que haga algo (le dan datos) o van a ser receptoras de datos
- Se puede usar tanto el análisis como para el diseño.
    - Análisis: desde una visión del sistema y cómo interactuar con el usuario y se define que se quiere del aplicativo teniendo en cuenta lo que se hace hoy. Se ve como el sistema interactúa en su contexto.
    - Diseño: Se enfoca más en la forma de hacer las cosas. Se ve como la aplicación informática interactúa en su contexto, tanto con terceros o con alguien de la organización que se contacta con terceros . Se ve mejor como va a ser la realidad de la aplicación.
- El nombre de la aplicación es importante ya que define el alcance. Ej: No es lo mismo hablar de un sistema de gestión(una actividad más amplia) que un sistema de registro (actividad más específica).
- Se ven las terminales/entidades, el nombre de la aplicación, flujos de datos
- Lo ideal es pensar cómo nace, se desarrolla y finaliza el negocio.





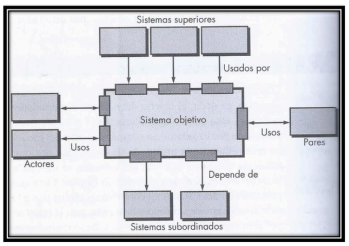
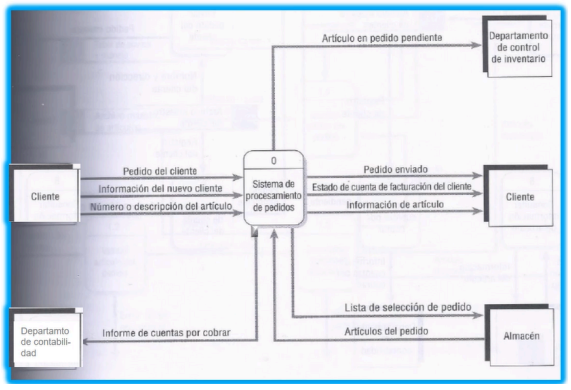
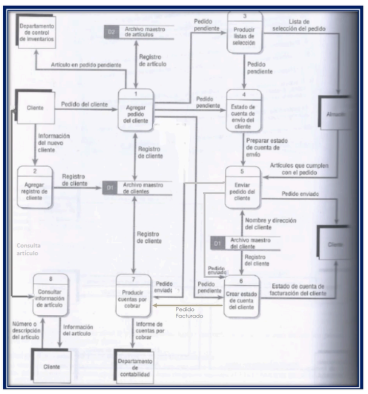


Diagrama de Contexto: Se separa la relación del sistema objeto con los sistemas superiores y subordinados, aplicativos pares y actores.

1. DFD NIVEL 0

Gracias al diagrama de contexto sabemos las entradas y las salidas del sistema (son constantes, son un dato de ese contexto). En esta instancia vamos a ver por primera vez cómo va a funcionar la aplicación por dentro para responder a la necesidad del negocio que nos da el contexto.
Para saber como hacer esto lo primero es ver **cual es la fuente y eso abrirlo en subsistemas** (y en los siguientes niveles cada subsistema se va a trabajar como pequeñas aplicaciones).
También vamos a analizar cuales son las funciones del negocio. A partir de las salidas y entradas que tenemos, hay que ver qué funciones generan esas salidas. Además hay que identificar los procesos y en qué almacenes van a incorporar los datos y quien va a necesitar esos datos (entidades).





↳ Aca se ve como se pasa de un diagrama de contexto a uno de flujos de nivel 0.
Solo se manejan archivos maestros. No se manejan archivos transaccionales en este nivel para no complejizar el diagrama. Estos no son procesos ejecutables, necesitan un grado de apertura con funciones más específicas. Esto lo haremos en los niveles siguientes de los DFD.

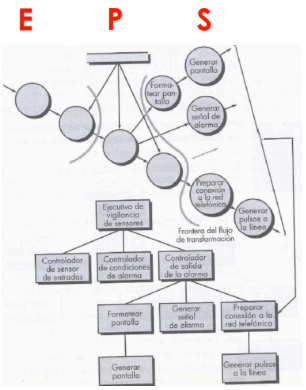
2. DIAGRAMAS HIJO (DFD1, DFD2...)

Se deben ver todos los flujos que se veían en el DFD0. Hay que explotar cada uno de los procesos manteniendo el contexto.
Para no hacer muy extenso el gráfico, se suele trabajar con cada proceso por separado. Ej: explotamos el proceso 1 del gráfico anterior: Agregar pedidos del cliente.
Se pueden ver detalles que antes no, como por ejemplo qué pasa si no se encuentra el cliente o el artículo.
Pero también hay cosas que todavía se deben desarrollar más. Por ejemplo: el archivo maestro de clientes debe estar dividido en los datos de la cuenta corriente por un lado y los datos básicos por otro.
Lo mismo para el registro de artículos. Debería dividirse entre bajas, inmovilización, actualización de stock.

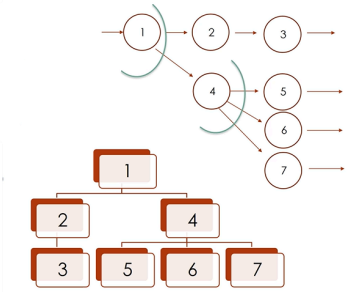
DIAGRAMA DE LLAMADO RETORNO

Una vez que tengamos el DFD → se pasa a la instancia de armar este diagrama para la instancia de diseño.
Tenemos que visualizar los DFD y mapear los flujos.

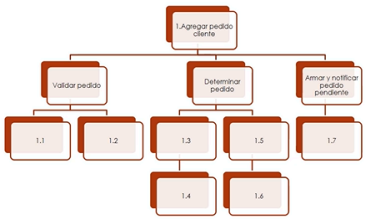
<u>Tipos de flujos:</u>

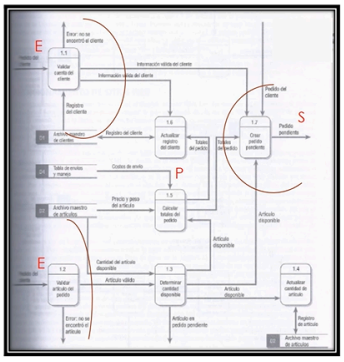


- **De transformación**
  - Identificamos las entradas, los procesos y las salidas de los flujos
  - Los procesos se dividen en 3: gestión de entradas, de procesos y de salidas
  - Luego se van a ir incorporando cada uno de los nodos dentro de la estructura. Ej: si tenemos el flujo de gestión de necesidades de capacitación vamos a tener un nodo de relevamiento de necesidades, otro de planificación de eventos (que se divide en un nodo de planificación/logística de evento y otro de ejecución del evento). El relevamiento sería la entrada, la planificación/logística el proceso y la ejecución la salida.
- **De transacción**
  - En vez de ver al DFD desde una visión de Entrada-Proceso-Salida aca lo vemos como líneas de procesos.
  - Se identifican los nodos de toma de decisión, que por ende generan nuevas líneas de procesos
  - Se trabaja con cada línea por separado para así poder generar el diagrama de llamada retorno.



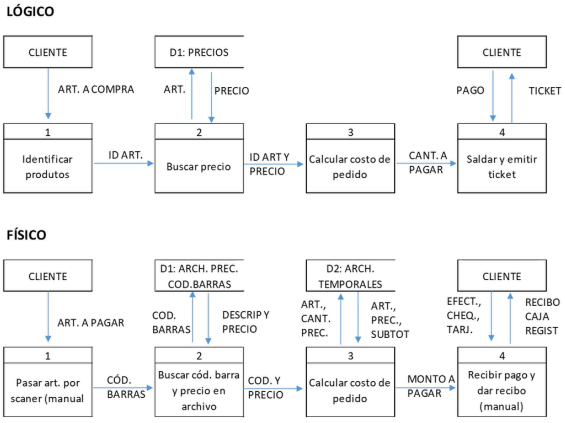
ANÁLISIS Y DISEÑO ESTRUCTURADO DE SOFTWARE





3. DIAGRAMAS/DFD FÍSICOS

Tanto el DFD actual como el esperado son herramientas de análisis
Mientras que en el diseño se usa un DFD lógico propuesto y un DFD físico implementado
El diagrama físico le suma al esquema lógico cuestiones como el hardware,software,archivos de consulta de datos, archivos donde guardaremos los datos , aplicativos, herramientas de hardware para el ingreso o egreso de datos, etc.



DFD ACTUAL | DFD ESPERADO | DFD LÓG. PROPUESTO | DFD FÍSICO IMPLEMENT.

ANÁLISIS DISEÑO

(VIDEO 6)

DICCIONARIO DE DATOS

Es un **repositorio** (registro metódico de informaciones sobre una o diversas materias) en el que se recogen, según un orden determinado, los datos con los que vamos a trabajar en nuestra aplicación. No se usan los datos propiamente dichos sino que con las denominaciones, significados en su contexto, características y ordenamiento metódico. Se guardan los metadatos (datos que guardan los datos).

El repertorio se usa como un elemento de referencia permanente. Es el lenguaje con el nos vamos a entender dentro del software que vamos a diseñar y así evitar inconsistencias y ambigüedades (ej: si un número es entero o puede tener decimales).

Va a servir para generar un lenguaje común y ser un soporte tanto en análisis, diseño y codificación de la aplicación (en el tratamiento de variables, en la planificación de pruebas y en la identificación de errores).

Muchas veces los errores tienen que ver con la forma que el código hizo el tratamiento del dato y eso puede estar dado por la característica de ese dato que está explicitado en el diccionario.

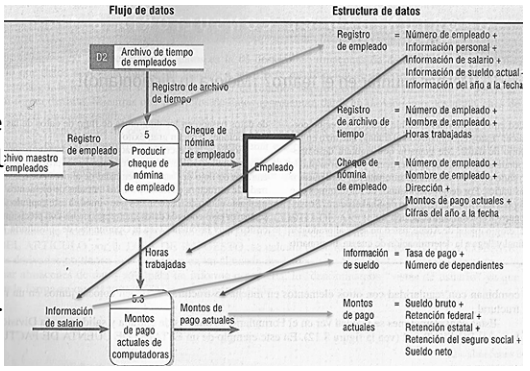
Diccionarios automatizados (gestionados automáticamente):

- Permiten hacer referencias cruzadas con los diferentes aplicativos.
- Se usan solo las referencias cuando se hace algún ajuste para entender cuál puede ser el impacto que esa modificación en el dato puede tener
- Son necesarios en sistemas muy extensos
- Permite saber el significado, la denominación, que programas manipulan los datos, etc. Todo lo referido a los datos de cada dato.

Un buen diccionario de datos:

- Permite validar la integridad y precisión de los DFD
- Sirve como documentación en el desarrollo.
- Se exponen decisiones de los datos
- Permite evaluar la claridad de esas definiciones
- Evita la redundancia al tener los datos en un solo ámbito
- Soporte relevante para definición de base de datos
- Punto de partida para las pantallas de informes o reportes

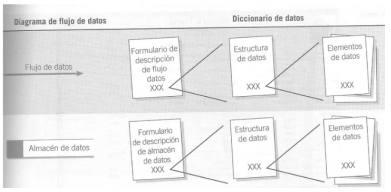
Repositorio de datos de proyecto: Ámbito donde se guardan los datos propios que hacen al proyecto → diccionario de datos, datos y procedimientos asociados, flujos, registros, requerimientos, almacenes, datos de entrada y de salida, entregables y toda otra información del proyecto.



¿Cómo elaborar el diccionario de datos?

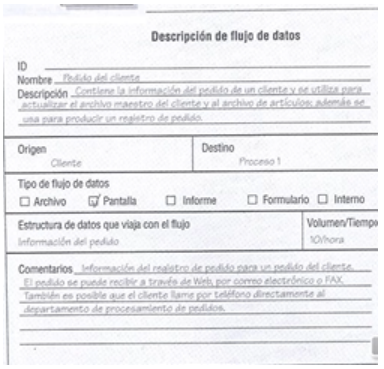
- ➔ Puede hacerse antes o después del DFD. Lo recomendable es que sea en simultáneo y ver consistencia de datos al bajar de nivel del DFD.
- ➔ Se realiza en la etapa de desarrollo con el avance del proyecto.
- ➔ Identificamos los flujos de E/S teniendo en cuenta:
  - ◆ DFD Lógico: contiene información cercana al negocio (datos). Contenido de los formatos (información).
  - ◆ DFD Físico: Contenedor de la información. contiene lo relacionado al diseño/formatos. Ej: estados de cuentas, facturas.
  - ◆ Almacenes
  - ◆ Procesos

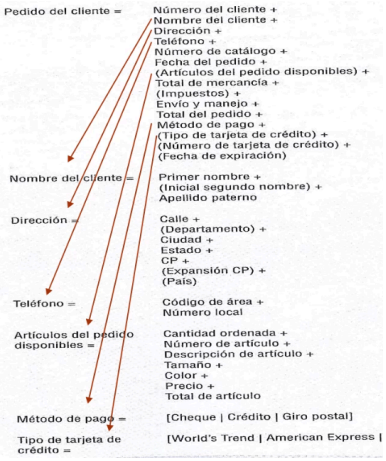
- ➔ Detalle aportado por el usuario (tienen que ver con el negocio)
- ➔ Formato de los flujos (ej: Formulario de E/S). Elementos que se piden o muestran a los usuarios que especifica la forma que se quiere ver el dato.
- ➔ Datos temporales (no se resguardan), permanentes (maestros) y semi permanentes.
- ➔ Tiene que haber consistencia entre los niveles de abstracción de los distintos DFDs, mantiene la correlación de los elementos y lógica del flujo



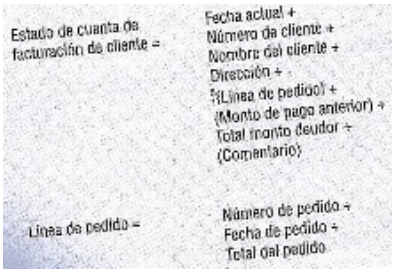
<u>Pasos:</u>

1. **Definir los flujos de datos para trabajar individualmente con cada uno:**
   - 1.1. Del DFD tomamos los flujos de E/S (flechas) y almacenes
   - 1.2. Tratamos de generar un documento que establezca un número y nombre del diagrama a cada flujo, descripción (Entidad de entrada y entidad de salida). Da pistas para el tratamiento de canales y flujos y cada uno va a tener un tratamiento.
   - 1.3. Determinar el origen y dónde se va a almacenar (almacén, terminal, etc).
   - 1.4. Determinar el método de entrada (por pantalla, lector, archivo o formulario).
   - 1.5. Estructura de datos con nombre propio
   - 1.6. Volumen / Tiempo → demandas
   - 1.7. Comentarios

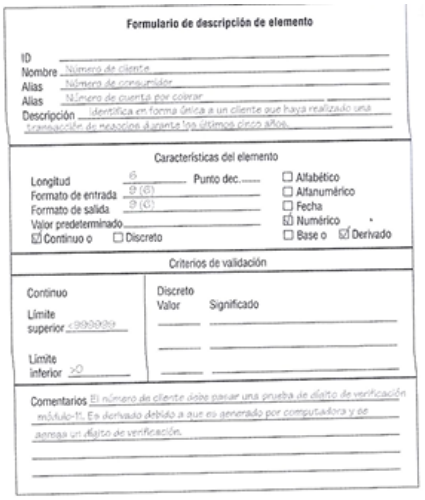




2. **Describir la estructura de datos del flujo seleccionado**
   - 2.1. Mayor apertura del flujo para ver por qué datos está compuesto y de donde proviene cada uno de los datos. Ej: pedido de clientes está compuesto por un nombre, teléfono, fecha, etc.
   - 2.2. Seguir desagregando y ver cómo se compone el nombre del cliente, el teléfono, etc. Se normaliza la base de datos. Es una estructura de datos lógica, una estructura de negocio.



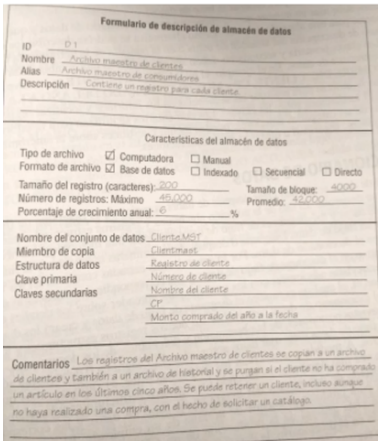
3. **Describir los elementos de datos de esa estructura**
   - 3.1. Identificar campos claves para búsqueda en base de datos:
     - 3.1.1. Códigos que permitan establecer un estado a un maestro que es necesario para la gestión del programa.
     - 3.1.2. Código de las transacciones que manipulan el dato de las transacciones.
     - 3.1.3. Conteo de datos repetitivos
     - 3.1.4. Contraseñas
     - 3.1.5. Tipos de usuarios
     - 3.1.6. Estructurar datos de salida y su referencia en cómo debe ser
   - 3.2. Incorporar información sobre cómo se manipula el dato
     - 3.2.1. Describimos la característica propia de cada dato
     - 3.2.2. Nombre de los elemento
     - 3.2.3. Alias (puede tener connotación distinta en distintas partes del programa)
     - 3.2.4. Descripción - si es base o derivado de algún otro proceso
     - 3.2.5. Longitud (valores máximos y mínimos que puede tomar el dato)
     - 3.2.6. Tipo de dato
     - 3.2.7. Formato específico de entrada y/o salida. Ej: cant decimales.
     - 3.2.8. Valores predeterminados
     - 3.2.9. Criterio de validación
     - 3.2.10. Otros comentarios



4. **Identificar los contenidos de los almacenes de datos de ese flujo**

4.1. Vamos trabajando en la descripción de los almacenes de datos que van a ir agrupando todos los elementos que contienen las estructuras de datos que definimos, siguiendo los flujos que tenemos.

4.2. Definir los datos del almacén
- 4.2.1. Nombre y numero (misma nomenclatura que en el DFD)
- 4.2.2. Alias
- 4.2.3. Descripción
- 4.2.4. Tipos de archivos donde se van a registrar
- 4.2.5. Formato
- 4.2.6. Tamaño máximo de cantidad de registros (para saber que motor de base de datos usar y cómo los voy a gestionar)


Formulario de descripción de almacén de datos

ID D1
Nombre Archivo maestro de clientes
Alias Archivo maestro de consumidores
Descripción Contiene un registro para cada cliente

Características del almacén de datos

Tipo de archivo ☑ Computadora ☐ Manual
Formato de archivo ☑ Base de datos ☐ Indexado ☐ Secuencial ☐ Directo
Tamaño del registro (caracteres): 200 Tamaño de bloque: 4000
Número de registros: Máximo 45,000 Promedio: 42,000
Porcentaje de crecimiento anual: 6 %

Nombre del conjunto de datos: ClientesMST
Miembro de copia: ClientesMST
Estructura de datos: Registro de cliente
Clave primaria: Número de cliente
Claves secundarias: Nombre del cliente
CP
Monto comprado del año a la fecha

Comentarios: Los registros del Archivo maestro de clientes se copian a un archivo de clientes y también a un archivo de historial y se purgan si el cliente no ha comprado un artículo en los últimos cinco años. Se puede retener un cliente, incluso aunque no haya realizado una compra, con el hecho de solicitar un catálogo.

(VIDEO 7)

ANÁLISIS Y DISEÑO ORIENTADO A OBJETOS

- ➔ Es una metodología que define un método para poder construir una aplicación informática.
- ➔ No están tan marcadas las fases del ciclo de vida del análisis y diseño
- ➔ Se interpreta la realidad a partir de sus objetos
- ➔ Permite resultados rápidos y llegar con facilidad al detalle
- ➔ Es muy útil en:
  - ◆ Sistemas complejos: ya que permite encapsular el dato y modularizar (resolviendo de a pequeños problemas).
  - ◆ Sistemas de negocios en constante evolución y adaptación: ya que es fácil rediseñar, ampliar y mantener el software.

ULM (Lenguaje Unificado Modelado)

- ➔ Estándar de trabajo ampliamente aceptado y utilizado
- ➔ **Mejora de la calidad**: en el producto, en su entrega, en los procesos
  - ◆ Permite dar respuestas rápidas con una asociación muy rápida entre el análisis y el diseño
- ➔ **Iteración**: trabaja en ciclos sucesivos desde el alto nivel hasta el bajo nivel con facilidad. Trabaja con un conjunto de diagramas predefinidos que reflejan el comportamiento y lo que se espera de la aplicación.
- ➔ **Reciclaje:** el concepto de clases implica el uso de estructuras predefinidas adaptadas a cada instancia que permiten reciclar dentro del contexto del lenguaje pero también entre proyectos.
- ➔ **Objeto**: es la instancia de una clase
  - ◆ Es la suma del dato y el código asociado encapsulado (haciendo que el impacto de un error no repercuta en otras partes del sistema)
  - ◆ Cada objeto es pensado como una parte del sistema con sus atributos y métodos
  - ◆ Ej: un martillo es una instancia de la clase herramientas
- ➔ **Clase**: es una plantilla/modelo para crear objetos
  - ◆ Personifica la estructura de datos (variable de tipo string)
  - ◆ Agrupa miembros con una similitud
  - ◆ Es la unidad de análisis y diseño
  - ◆ Cada clase está relacionada con entidades referidas al mundo real, interfaces (límites del sistema), entidades abstractas o clases de control.

Conceptos básicos:

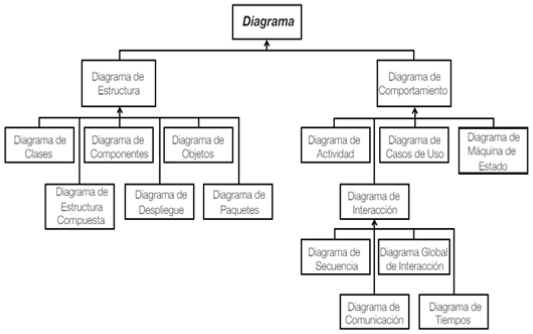
- ➔ **Objetos y clases:** A partir de interpretar la estructura (atributos y métodos) del objeto o de las acciones que realiza se va definiendo la clase.
  - ◆ Se parte del objeto y llegamos a la clase a partir de la categorización del objeto → Esa clase se insta para la creación de un objeto → Se retroalimentan.
  - ◆ A partir de la definición de atributos y métodos podemos reutilizar la instancia de una clase y convertirla en otro objeto.
- ➔ **Herencia:** capacidad de una clase (subclase/clase hija) para heredar atributos y métodos de otra clase (superclase/clase padre). Una clase puede extender o especializar el comportamiento de otra clase.

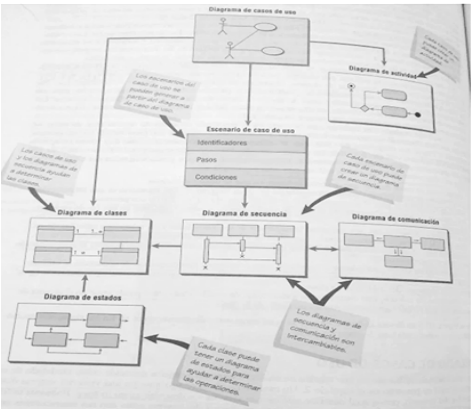
- Facilita la reutilización del código y la organización de la jerarquía de clases.
- Ej: si tenes una clase Vehículo con atributos y métodos básicos, como color, marca, acelerar, podes crear subclases como Autobús, Taxi, que heredan todas las características de la clase Vehículo, pero también pueden tener sus propios atributos y métodos específicos.
  Permite modelar con precisión el comportamiento de diferentes tipos de vehículos mientras reutilizas el código común.
- Existen diferentes métodos. Los métodos públicos son fácilmente compartibles, y los privados no se comparten.

➔ **Encapsulamiento:** Los objetos encapsulan (ocultan) sus métodos y atributos para otros objetos. Hay que tomar decisiones en cuanto a qué quiero mostrar de la clase.
- Las clases públicas pueden acceder a sus elementos desde cualquier método definido.
- Las clases protegidas heredan accesos de las clases padre-hijo.
- Las clases privadas (las más restringidas) no permiten el acceso de otros objetos y clases, solo desde objetos propios de esa clase.

➔ **Asociación:** Al identificar las clases se conocen los tipos de vínculos que existen entre las clases desde una visión de negocio. Es importante entender el vínculo, su restricción y el tipo de relación (si es 1-N, 1-1, etc).

➔ **Agregación:** Conjunto de clases que integran un todo que las puedo conectar. Que al afectar a una, afecto a todas.





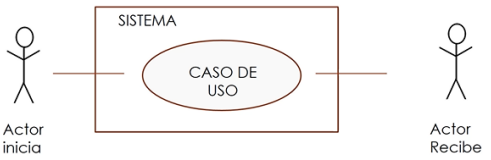
Diagramas básicos de ULM:

- **Diagramas de Estructura**
  - **Diagrama de clases (estático):** estructura arquitectónica de la aplicación donde identifico las clases, sus atributos y métodos y su interrelación (los casos de uso y los diagramas de secuencia ayudan a determinar las clases).
  - **Diagrama de objetos:** plantea el diagrama de clases listados en objetos de forma total o parcial.
  - **Diagrama de paquetes:** forma de subdividir en paquetes entregables según cuestión lógica del negocio.
  - **Diagrama de componentes:** forma de agrupar por criterios determinados componentes de la aplicación.
  - **Diagrama de despliegue:** Podemos hacer coexistir la parte lógica y física de la aplicación. Dónde va a correr determinadas partes de la aplicación.
- **Diagrama de comportamiento**
  - **Diagrama de casos de uso (estático):** Plantea los diferentes escenarios de los usuarios en los momentos de verdad, identificando las funciones asociadas al usuario. Los escenarios del caso de uso se pueden generar a partir del diagrama del caso de uso. Los escenarios tienen identificadores, pasos y condiciones. Tiene entrada, proceso y salida. Identifica quién va usar el evento.
  - **Diagrama de actividad (estático):** Exhibe el flujo de la información a través de las distintas actividades que realiza la aplicación. Muestra el ciclo y el desarrollo del escenario de usuario para cada uno de los casos de uso. Cada caso de uso puede crear un diagrama de actividad.
  - **Diagrama de secuencia (dinámico):** Plantea cómo los objetos a partir de sus intercomunicaciones van interactuando a lo largo del tiempo. Colaboración entre los objetos y clases. Cada escenario de caso de uso puede generar un diagrama de secuencia.
    Los diagramas de secuencia y comunicación son intercambiables).
  - **Diagrama de comunicación:** Donde los componentes y las clases se van comunicando entre ellas.

- **Diagrama de máquina de estado:** Diferentes estados de los objetos. Cada clase puede tener un diagrama de estados para ayudar a terminar las operaciones.

(VIDEO 8)

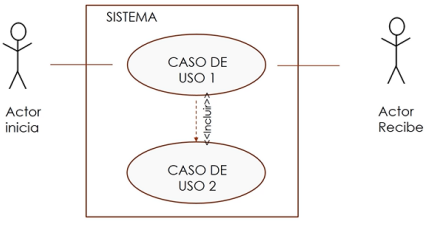
DIAGRAMA DE CASOS DE USO



Es el momento de verdad del actor frente al aplicativo. Utiliza el lenguaje ULM. Define un modelo lógico del sistema y muestra qué hace el sistema (sin mostrar cómo lo hace) desde una perspectiva de usuario. Proceso narrativo de la acción del actor hasta llegar a un resultado. Ayuda a pensar en el usuario. Pone foco en el camino feliz, la situación exitosa → Se trabaja sobre la normalidad excluyendo las excepciones (sin los "si no"). Detalla el alcance del sistema a través de las funciones de negocio. Los casos de uso se escriben en infinitivo. Solo se escriben los casos de uso relevante para el negocio. Ayuda a involucrarnos con el usuario tanto en el análisis como en el diseño, sobre lo que quiere y sobre lo que se le puede dar.

Componentes:

- **Actores:** de carácter genérico, áreas, personas, sistemas, dispositivos o clientes. Unos son iniciadores y otros reciben los resultados (pueden ser ambos). Identificar el tipo de actor, coloco una denominación, habilidades que tiene cuando gestiona la tarea, cual es el objetivo y su prioridad. El que inicia puede ser el mismo que lo recibe.
  - Recibe: Recibe el resultado del caso de uso
  - Principales: Dan datos al sistema y generan el evento. Alimentan el detalle para recibir el caso de uso y pueden recibir información. Pueden ser directos o indirectos (actúan a través de otros actores).
  - Secundarios: Técnicos y analistas. Son soporte al sistema.
- **Funciones**: que determinan el comportamiento del sistema que genera un producto tras su procesamiento. Es una forma de modularizar el trabajo mediante funciones de negocio.
- **Respuesta:** Producto que debe devolver el sistema hacia el actor.
- **Sistema**: Denominamos el sistema y establecemos las fronteras que separan los actores del caso de uso. Los actores están afuera.
- ➔ **Inclusión**: Desde un caso de uso convocar a otro caso de uso que se está garantizando habitual: Las operaciones habituales las hago una sola vez. Si otro proceso lo necesita también, lo invoca.
  Ej: tengo un caso de uso para registrar pedidos y tengo que hacer una validación de que el cliente exista.



- ➔ **Extensión**: Trabaja un caso específico → Divide la normalidad de las excepciones. Si no es algo común → extensión del caso de uso.
- ➔ **Generalización**: Hay determinados casos de uso que tienen características generales que son compartidas por otros casos de uso. Planteamos la idea de una herencia. En el caso de uso 1 levantas especificaciones del caso de uso general. Permite reciclar, estandarizar y normalizar.
  Ej: un proceso de compras y otro con particularidades de compra directa o compra mayorista o minorista que tienen especificaciones adicionales de la compra general.
- ➔ **Cohesión**: Se auto encierra en su propia lógica. Tiene toda la lógica propia para sacar un producto con fuerza funcional.
- ➔ **Independencia funcional**: Lo veo como una función mínima de funcionalidad. Fuerte hacia adentro.

El caso de uso se puede hacer a medida que conozco el negocio, realizo entrevistas, identifico los actores que puedan dar información, los procesos de negocio y sus reglas y las funciones de negocio.

Están interrelacionados los prototipos y el caso de uso. Ambos trabajar a la par:

- ➔ Prototipo → visual, muestra como el actor se encuentra con el paso a paso.
- ➔ Caso de uso → define paso a paso lo que tiene que definirse en la actividad

Los casos de uso pueden dividirse por **niveles** según las necesidades/criterios de negocio y del programador:

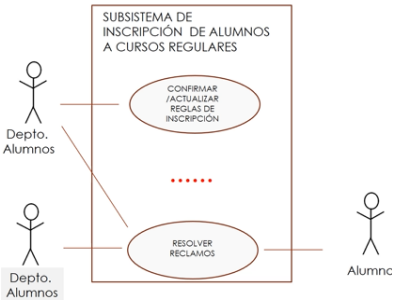
1. 1er nivel: puede ser muy genérico, que pueda estar o no asociado a proceso de negocio o pueden ser ya funciones de negocio. Según como la aplicación dará respuesta.
2. 2do nivel: voy bajando a diferentes departamentos (unidades propias del negocio). Veo los objetivos de cada departamento pero no las actividades de cada uno.
3. 3er nivel: cada departamento lo abro en caso de uso concreto de sus actividades.
4. 4to nivel: los principales los detallo en un requerimiento detallado. Descripción detallada de la actividad.

Los casos de uso se pueden agrupar bajo criterios de negocio.

(VIDEO 9)

CASOS DE USO

El caso de uso puede ser narrativo o puede ser gráfico. Narrativo → detalla al máximo que es lo que se hace.



1. **Nombre del caso de uso**:
   - Va a condicionar lo que pretendemos del caso de uso.
   - Punto de partida de entendimiento entre los distintos actores del desarrollo.
   - Debe ser significativo y relevante.
2. **Actores**: Son los participantes del momento de la verdad
   - Deben colocarse los actores que interactúan con el sistema.
     - Alumno regular (Padrón que puede inscribirse)
     - Departamentos de Alumnos (Habilitador)
3. **Responsables del caso de uso**: Definidores y responsables estructurales de la organización respecto del desarrollo y utilización de lo que se construya. Son los responsables del mantenimiento normativo. Ej: Departamentos de Alumnos
4. **Objetivo**: Descripción de la meta/pretensión del software. Ej: listado de solicitudes alumnos registrados.
5. **Desencadenante**: Situación o evento que permite que eso se ejecute. Puede venir de actores internos o externos. Ej: El departamento de alumnos que habilita la inscripción.
6. **Precondición**: Situaciones/condiciones que tienen que estar dadas para que los actores puedan desencadenar los eventos.
   - Que se haya ejecutado el caso de uso de Definir el curso
   - Que se haya ejecutado el caso de uso Determinar la oferta de matera
   - Que se haya ejecutado el caso de uso de Validación de alumnos
7. **Activador:** Activa al caso de uso. Se usa en aplicaciones que no están todo el tiempo en línea.
8. **Pasos:** Secuencia del actor y el sistema.
   - Debe ir todo (Mensaje, título, código materia, profesor, horario, tabla de salida)
   - Camino feliz → Todo lo otro van como excepciones.
9. **Requerimiento cumplido**: Lo que sucede cuando el requerimiento se cumple. Ej: solicitud de inscripción
10. **Poscondición:** Situaciones en la que se encuentra el caso de uso una vez que se realizaron los pasos y se cumplió el requerimiento. ¿En qué quedó la aplicación?. Ej: cómo queda el sistema luego de que el alumno se registró → una vez registrado envía el comprobante y no puede volver a registrarse.
11. **Salida:** como se sale de los casos de uso, cuáles son los exit en cada paso o en el final. En la salida que tiene que decir.
12. **Prioridad**: Se indica cuan prioritario es la aplicación. Me permitirá darle importancia que posea tanto para el desarrollo como para la puesta en producción.
13. **Excepciones**: Todo lo que no es el camino feliz. Puedo llamar a otro proceso que realice la excepción.
14. **Disponibilidad**: ¿Cuándo va a estar disponible para los usuarios? ¿Siempre o en momentos?
15. **Frecuencia de uso**: ¿Que tanto se va a usar?. Nos permite visualizar cuestiones tanto de sw como de hw.
16. **Interfaz**: Tanto con otros aplicativos como con usuarios
17. **Mensajes**: Tienen que estar bien definidos. Es importante evitar la ambigüedad y problemas futuros por mal entendimiento.
18. **Riesgo**: Cuan riesgoso es el caso de uso en términos de negocio. Sirve para determinar los niveles de seguridad que se van a necesitar.

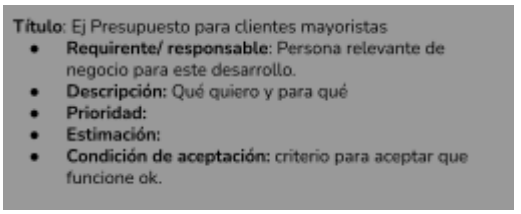
19. **Magnitud**: Sirve para manejar cuestiones de performance. Ej: Cant de actores, de información, etc

(VIDEO 10)

HISTORIA DE USUARIO Y DIAGRAMA DE ACTIVIDAD

Historias de usuarios

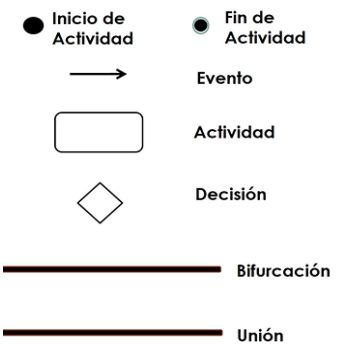
- ➔ Utilizadas en las metodologías ágiles
- ➔ Descriptivo pero corto → más acotado que el caso de uso
- ➔ Necesidad concreta del servicio que se pretende que surge de conversar sobre el problema y el esfuerzo
- ➔ Características a considerar:
  - ◆ Independiente. Que pueda concluir en un entregable (fácil de desarrollar, entregar e implementar).
  - ◆ Negociable
  - ◆ Aporta valor al negocio
  - ◆ Estimable: Definido en horas hombre/presupuesto financiero
  - ◆ Pequeño: Manejable en 15 días
  - ◆ Testable: Cond. de aceptación

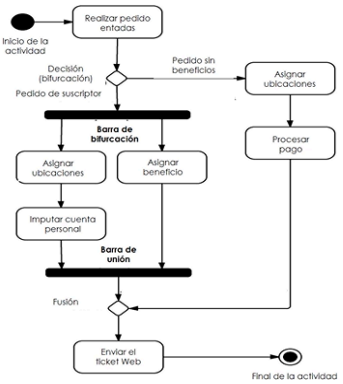


- ➔ Ventajas de las historias de usuario
  1. Son necesidades concretas y entendibles
  2. Fuente del requerimiento externa y conoce el negocio (usuario)
  3. Trabajo colaborativo
  4. Estimación de esfuerzo

Diagrama de actividad

- ➔ Mapeo de pasos que va desarrollando con la secuencia de pasos manuales y automatizados.
- ➔ Se utiliza para para diagramar algunos casos de uso, aspectos específicos de alguna parte de un caso de uso u otros usos (cualquiera que sea necesario).
- ➔ Se usa para ayudar a comprender, mostrar flujos complejos, mostrar distintos eventos que se van produciendo en este paso a paso de los casos de uso, para presentarlo al usuario si hay alguna cuestión compleja explicándole cómo funciona y también para ser más claro en lo que viene después en el proceso de construcción.
- ➔ Se va a usar cuando la dificultad del sistema amerite plasmarlo en este gráfico. No se usa siempre
- ➔ Se observa el inicio, el fin. las acciones, los nodos iniciales, finales y de toma de decisión y el flujo de control.





Pasos:

1. Identificar e Individualizar las actividades
2. Secuencialidad o paralelismo de las actividades. Qué ocurre primero y qué después
3. Identificar la toma decisión y el efecto que produce

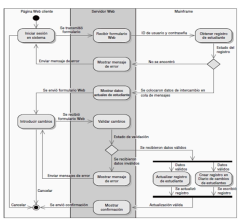


Diagrama de carril

Es una variación del diagrama de actividad.
Puede mostrar ambientes o actores.
Permite trasladar un diagrama de actividad a un ambiente físico o de negocio.

DIAGRAMA DE SECUENCIA Y DIAGRAMA DE CLASES *(VIDEO 11)*

DIAGRAMA DE SECUENCIA

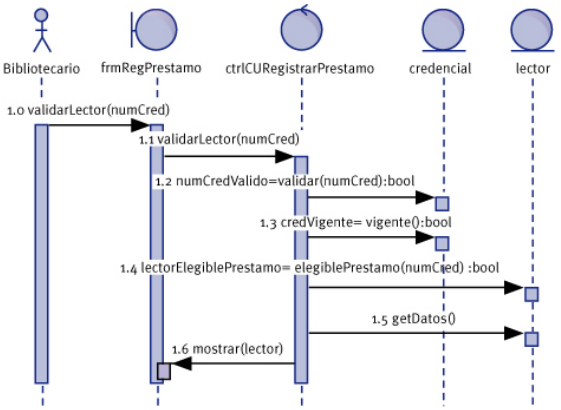
Permite ver cómo se suceden e interactúan los actores y las clases a lo largo del tiempo. Se vinculan con otras clases. Nacen y mueren. Permite plasmarlo de una manera más dinámica.
Permite modularizar la aplicación. Módulos y clases con el mismo nombre.
Métodos → encapsular y poder reciclar.

Componentes:

1) **Actor externo:** no se vincula con el sistema si no es a través de las interfaces. Se identifica a la izquierda del diagrama 
2) **Clases de interfaz:** no se vincula con los datos si no es a través de los organizadores. Nexo con usuario y aplicación. Conecta con las clases de control (Pueden ser más de una interfaz) soportando la comunicación con el exterior. 
3) **Clases coordinadoras (estáticas):** hace que los otros hagan. 
4) **Clases entidad:** es la base porque maneja el concepto del dato y sus métodos/atributos. Es la base de datos 
5) **Rectángulos verticales:** periodos de vida de la clase/objeto.

Se busca que se envíen mensajes entre las clases:

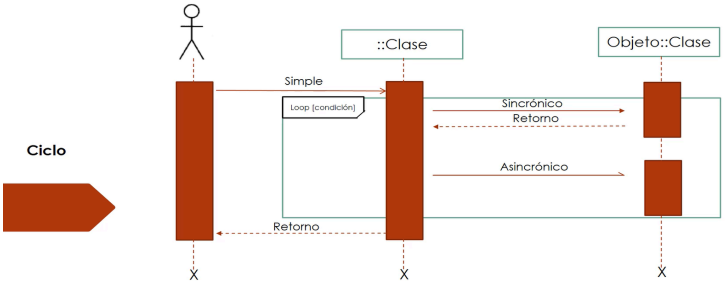
- Sincrónicos → esperan una respuesta inmediata. Se reflejan en una flecha
- Asincronico → realizan un proceso y no esperan una respuesta inmediata. Se reflejan con una flecha cortada.



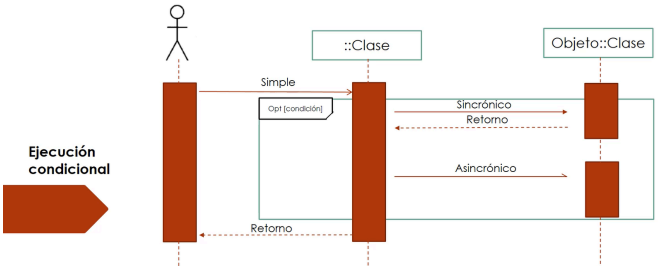
Hacia abajo se muestra la secuencia de trabajo en el tiempo.
Hay que comprender cada módulo en términos de entrada, proceso y salida. Hacer que cada módulo sea un problema individual.
Debe desarrollarse en conjunto con el diagrama de clases.

Elementos del diagrama de secuencia:

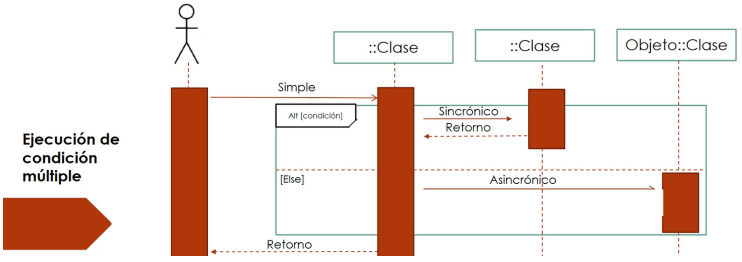
- **CICLO:** Se da el ciclo mientras se cumpla la condición. Me demuestra las vigencias de los objetos bajo esa condición.



- **CONDICIÓN (OPT):** Muestra el recorrido de la aplicación cuando se da esa condición.

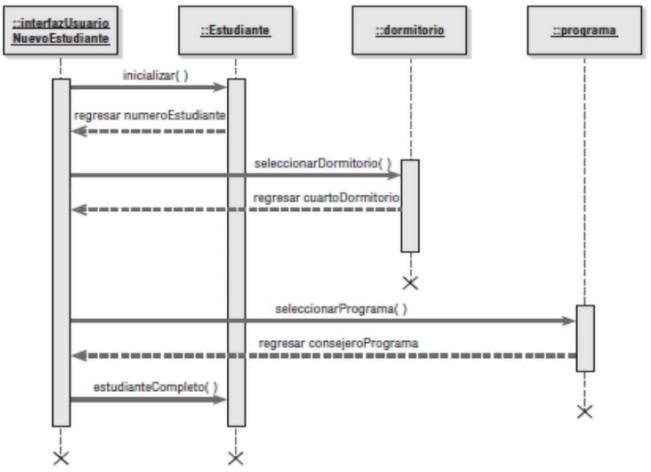


- **CONDICIÓN MÚLTIPLE (ALT):** if, else



Se puede hacer un loop del proceso con la condición.

- Condición de si:
  ALT()
  ELSE
- Opciones:
  OPC ()



Información a tener en cuenta para la construcción:

- Los usuarios se vinculan con interfaces de usuario (clase)
- Hay que identificar los actores en el caso de uso
- Las clases de entidad no se vinculan directamente con la interfaz de usuario.
- La clase entidad trabajan sobre la persistencia del dato, como accedo al dato y lo modifico
- La interfaz de usuario solo actúa en el usuario (no gestiona, ni se relaciona con otras clases)
- Las clases interfaz se ocupan de la presentación ante el usuario de la aplicación.
- Esas interfaces ingresadas y recepcionadas deben ser manipuladas por una clase de control que arregle la vinculación con otras clases de control y de entidad
- Cada caso de uso debe tener, al menos, una clase de control que gestiona las reglas de negocio
- Las clases de control se ocupan de gestionar las capas del negocio.
- Es importante revisar si la clase de control podría estar dentro de la clase entidad, mejorando la reutilización. Ej: una consulta.

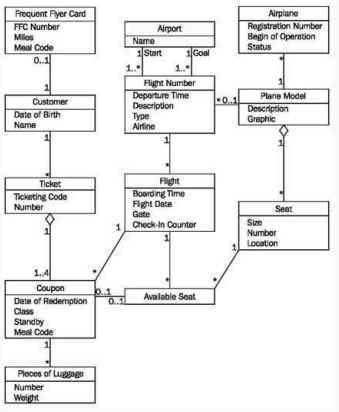
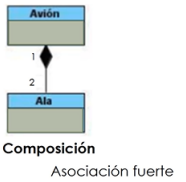


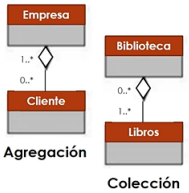
DIAGRAMA DE CLASES:

Las clases como datos (entidad-relación)

Es un tipo de diagrama de estructura estática que describe la estructura de un sistema mostrando las clases del sistema, sus atributos, operaciones o métodos y las relaciones entre los objetos. Hay distintos tipos de asociaciones entre clases:

- 1-1
- 1- 1/M
- 1 - 0/M
- 1 - M
- M-M
  - De muchos a muchos: Se tiene que generar una clase de vinculación entre esas 2 clases.
  - **Reflexiva:** Se retroalimenta. Ej: Un supervisor tiene un supervisor y un subordinado.
- **Generalización/Especificación:** heredan lo que viene de arriba
- **Composición**: Una entidad es dominante fuerte (diamante cerrado) sobre otra. Si elimino el dominante, elimino el débil; si elimino el débil, el dominante sigue.
  Se pone la punta del diamante del lado dominante.
- **Agregación**: Son codependientes. Si elimino uno, elimino los dos (Diamante abierto)
- **Colección**: Relación débil. Si se elimina un libro, no se borra la biblioteca (Diamante abierto)





(VIDEO 12)

DISEÑO DE INTERFAZ

Primer momento de verdad frente al usuario. Es el diseño de interfaz de usuario y aplicaciones.

- **Interfaz de aplicación:** Debe tener una comunicación clara con el otro aplicativo. Procesar correctamente el input de información.
- **Interfaz de usuario:** Primera impresión al comportamiento de lo que vé sin importar lo que procese por la regla de negocio. Es el momento de la verdad. Donde el usuario ve si se siente cómodo o no con el sistema. Evitar la propagación de errores.

Principios de la interfaz de usuario:

1. **Dejar el control al usuario**
    a. El usuario no se ve manipulado por una aplicación, sino al revés.
    b. Aportar valor a la tarea.
    c. Debe saber dónde quedó la transacción cuando haya error.
    d. Habilidad de uso, que lo convierta en un experto.
    e. No tenga a la vista tecnicismos del sistema.
    f. Permitir la interacción con la pantalla (flexibilizar).
2. **Reducir la carga de la memoria del usuario**
    a. Establecer en pantalla, atajos intuitivos y no transacciones en código alfanumérico.
    b. Usar metáforas de la vida real
3. **Conocer el contexto de la interfaz**
    a. Conocer en qué ambiente de trabajo va a coexistir.
    b. Conocer el perfil del usuario.
    c. Dispersión de edades.
    d. Ambiente organizacional (símbolos, colores).
    e. Uso de los contrastes.
    f. Tipo de salida.
    g. Cantidad de usuarios
4. **Desarrollar una interfaz "Consistente"**
    a. Debe asociarse a la tarea.
    b. Otras aplicaciones que coexisten.
    c. Acoplarse con los colores.
5. **Usar el lenguaje adecuado**
    a. Comandos, menú. Tener en cuenta el lenguaje del negocio para diseminar errores.
6. **Diseñar, validar y construir**
    a. Avanzar con el prototipo más afinado a la construcción para garantizar la aceptación del usuario.
7. **Elegir la metáfora adecuada**
    a. Formato y simbología de la interfaz.
    b. Como se presenta la interfaz. Que sea acorde al usuario, al trabajo y a la organización.
    c. Elegir una plataforma que se adapte a la necesidad del negocio más que por lo que esté de moda.
8. **Gestionar tiempo de respuesta**
    a. Para prevenir la ansiedad del usuario.
    b. Se cumplen los tiempos de respuesta → Informar los tiempos.
9. **Manejar el error, ayudar y permitir el acceso**
    a. Impedir el error que generamos nosotros.
    b. Ayudar frente al problema.
10. **Una decisión estratégica de implementación**
    a. Qué quiero lograr con ello.
    b. ¿Romper prácticas pasadas (cambios) o mantener criterios para que sea más sencillo de usar?

<u>Pautas para el diseño de SALIDAS</u> (mensajes, archivos, pantallas visuales, datos descargables, sonidos, alertas auditivas, actualizar registros, interacciones con otros dispositivos, respuestas táctiles):

1. **Con un propósito previsto:** que el sistema genere valor agregado a la tarea, que sea significante la implementación.
2. **Ajustadas a los usuarios:** en función a tareas y responsabilidad jerárquica y el correcto manejo de info.
3. **Dónde, cuándo y cuánto se necesita**
    a. Para determinadas transacciones o tareas.
    b. Que la información sea lo justo y necesario
4. **Método correcto - Selección de la tecnología**
    a. Forma de salir según costo de salida y la tarea
    b. Flexibilizar la información.
    c. Tiempo de información.
    d. Tiempo de recuperación de la información

5. **Discriminando salidas internas y externas**
    a. Riesgos de manejo de información en cada una de las salidas y su impacto.
    b. Tener en cuenta las condiciones legales.
6. **Entender cómo la información impacta en la organización:** No todos los usuarios están acostumbrados de visualizar la información de la misma forma.
    a. Ordenamiento de la información
    b. Valor de la información
    c. Gráficos/Visuales
    d. Capacitación de los usuarios (minimizarlo)
7. **Salidas impresas y por pantalla**
    a. Tiempo de resguardo
    b. Quién utilizará la salida
    c. Velocidad esperada de respuesta
    d. Frecuencia de uso
    e. Normas especiales
    f. Costos de la salida
    g. Las salidas impresas tienen un costo de almacenamiento físico.
    h. Requisitos humanos o ambientales.
    i. Tipos de información
    j. Organización de los datos y armar un modelo de informe.
    k. Número de página, encabezados, agrupación lógica de información.
    l. No afectar la información.
    m. Demostrar si muestra toda la info
    n. Atajos
    o. Facilidad de comprensión
    p. En pantalla → tableros de control

Pautas para el diseño de ENTRADAS:

1. **Formularios (concepto administrativo):** Es el origen de los datos, guía el comportamiento del usuario. Completo y útil.
2. **Fácil de llenar:** Amigable a la vista, ágil con el proceso de llenado, de arriba a abajo y de izquierda a derecha, con encabezado y firma de usuario.
3. **Se asocia con la tarea y permite cumplir el propósito**
4. **Evitar el error- evitar la propagación:** totales de control
5. **Mantener atractivos los formularios/pantallas para atraer el uso.**

Para ello se debe:

- Ser simples → mostrar sólo lo necesario
- Permitir movimiento - sin perderse, incluir ayudas de contexto.
- Ser atractivas y agradables según el contexto (no tantos clicks)
- Apelar a íconos e interfaz gráfica de usuarios (GUI)
- Ser consistente para el uso de controles
- Utilizar colores y tipografía con criterio (no más de 7 colores). El color ayuda a hacer foco. El tamaño de letra y tipografía tiene que ser acorde al usuario que lo va a utilizar.

(VIDEO 13)

PRUEBA DEL SOFTWARE

Revisar errores antes de salir en producción. Si sale a producción con errores el usuario puede sentirse insatisfecho y el equipo de desarrollo deberá volver hacia atrás y puede llevar altos costos.

Las pruebas se realizan en paralelo a todas las fases del desarrollo para prevenir aumento de costos, reducir los errores en cascada. Genera calidad en el proyecto. Los casos de prueba deben documentarse.

Una buena prueba y exitosa es aquella que encuentra errores. Detectan la existencia del error, no cual es el error. Se prueba el ejecutable y el programa fuente.

Técnicas de prueba del software

- ➔ Son garantía de calidad acompañadas de auditorías, y control de calidad y factibilidad.
- ➔ Se realiza una revisión final de: **especificaciones, diseño y codificación**
- ➔ Forman parte de la **verificación y validación del software:**
    - ◆ Verificación: actividades tendientes a comprobar que el software cumple con una función específica. ¿Se construyó el programa correctamente?

- Validación: válido si el sistema se está realizando en base a los requerimientos implícitos y explícitos. ¿ Se construyó el producto correcto?

Prueba del código fuente → descubrir la máxima cantidad de errores evitando generar defectos.
"El testing puede probar la presencia de errores pero no la ausencia de ellos".

Características:

- En la parte de diseño debe probarse el código fuente para descubrir la máxima cantidad de errores para evitar generar defectos.
- La prueba la debe realizar quien no haya desarrollado o codificado el software. Pueden ser potenciales usuarios o un área de testing.
- Se deben ir desarrollando las pruebas mientras se va codificando el software
- Regla de pareto: En el 20% del código se encuentra el 80% de los errores.
- Se debe ir de abajo hacia arriba. De lo más pequeño a lo más grande → unidad, método, modulo, clase
- Pruebo y después depuro.
- No existen las pruebas exhaustivas → se prueban las que tienen más probabilidad de ocurrencia o mayor impacto.
- Rastrear el código hacia las decisiones que llevaron al código. Necesito ir para atrás.

Características de una buena prueba:

1. Elevada probabilidad de encontrar un error
2. No debe ser redundante → no deben tender a probar lo mismo.
3. Ejecutarse por separado para no solapar los casos de prueba
4. La mejor de su clase (Más casos reales, con menos costos)

Principios de las pruebas

1. Todas deben ser rastreables hasta los requisitos
2. Son progresivas
3. No es posible realizar pruebas exhaustivas (es imposible probar todo por el costo y tiempo)
4. Deben planearse anticipadamente

Estrategia de prueba: Plan generalmente elaborado sobre el fin de cada etapa de análisis. Objetivos:

1. Guiar las pruebas
2. Definir cuándo realizarlas y qué tipos usar
3. Establecer cuánto esfuerzo, tiempo y recursos se va a requerir



Deben incorporar:

- Planificación de la prueba
- Procedimientos de prueba
- Ejecución
- Recolección y evaluación de resultados
- Actividades de depuración

Las pruebas son inversas a las actividades de desarrollo.

- Test unitarios → pruebas para unidades pequeñas y de cada módulo individual.

Prueba de integración: Ensamblaje de los distintos módulos entre sí probando las interfaces. Se utiliza un **enfoque incremental de integración.**

1. Descendente: desde arriba (cúpula) hacia abajo (empleados)
    a. Profundidad (en ramas)
    b. Anchura (Incorporando el siguiente nivel)
2. Ascendente.
3. Pruebas sandwich: combinación de ascendente y descendente.

- **Prueba de regresión:** Utilizan pruebas que ya se realizaron, así no se tienen que inferir otros datos.
- **Pruebas de humo:** Se hacen de manera urgente porque forman parte de un código crucial que necesito probar rápidamente para no acarrear el error.

- **Pruebas alfa y beta:** Las alfa son en ambiente de producción mientras que en las beta les dan a algunos usuarios para probar el sistema.
- **Pruebas de seguridad:** Protección ante accesos indebidos. Intenta entrar y vulnerar el sistema.
- **Pruebas de resistencia:** Probar cómo reacciona ante exigencias mayores a las normales. Se castiga al sistema con muchos accesos al mismo tiempo para ver si aguanta.
- **Pruebas de desempeño**: Se hace para ver la variabilidad de las respuestas.
- **Prueba orientación objeto:** voy creciendo hasta probar la clase.
- **Prueba del sistema:** se comprueba la totalidad del sistema.
- **Pruebas de recuperación:** verificar que no caiga todo el sistema ante una falla o que se recupere en un lapso razonable. Se fuerza el fallo.
- **Pruebas de despliegue/configuración:** Para sistemas que deben implementarse en distintas plataformas y sistemas operativos, ej: páginas web. Se evalúan procedimientos de instalación, programas instaladores y documentación.

Como no es posible realizar una prueba exhaustiva debemos deber una visión estratégica:
Identificar y probar módulos críticos. Los módulos críticos son los que se relacionan con varios requisitos del software, tienen un mayor nivel de control, es complejo, propenso a errores, tiene gran exigencia de rendimiento.

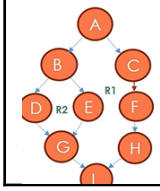
Pruebas de integración orientadas a objetos: En lugar de probar un módulo, prueba una clase encapsulada y por el comportamiento de estado de la misma. Existen dos estrategias:

1. **Basada en hebras:** Integra el conjunto de clases requeridas para responder a una entrada o evento del sistema. Cada hebra se prueba por separado. La técnica de regresión se utiliza para que no haya efectos colaterales.
2. **Basada en uso:** Prueba las clases que usan pocas clases del servidor, las independientes y luego se examinan las clases dependientes.

Pruebas de unidad:

- ➔ **Caja blanca:** Revisa el módulo individual. Su código fuente a través de pruebas de trayectorias, ciclos, estructura de datos, y de interfaz (info desde y hacia el módulo). Se tiene conocimiento interno del software y se ve el código por dentro. Se trata de agrupar el código en lotes/módulos para así probarlo. Se definen las rutas básicas por las que va el código. Cada nodo se va poniendo en una matriz. Se prueba cada nodo y se le va dando valores. Nos enfocamos donde vemos más errores.
- ➔ **Caja negra:** Existe un módulo impulsor que no es un programa que recibe un lote de prueba y devuelve los resultados. Los auxiliares responden al llamado del módulo. Programación que luego, se desecha. No se ve el código hacia adentro, solo se ven las entradas y salidas.

Métodos de prueba: Las pruebas se deben conducir de forma sistemática probando el soft desde 2 visiones:

CAJA BLANCA (LÓGICA INTERNA) Se conoce como se hace	**CAJA NEGRA (Cumplimiento de los requisitos)** Se conoce el qué se hace
Técnicas de pruebas de caja blanca Es un método de diseño de casos de prueba que usa la estructura de control y la lógica de sw bajo testeo para derivar los casos de prueba. Su objetivo es verificar el flujo de control y datos internos. ● **Prueba de caminos o trayectorias:** ○ Diseño de casos de prueba para medir la complejidad lógica del diseño estructural definiendo un conjunto de caminos de ejecución ○ Cada instrucción del componente se ejecuta al menos 1 vez ○ Complejidad ciclomática: Determina el límite superior de trayectorias independientes y el número de casos de prueba que asegure que cada sentencia se ejecute al menos una vez. ■ Nodos (círculos): instrucciones ● Nodos predicados (B): salen 2 o + flechas ■ Flechas: son enlaces ■ Regiones: áreas encerradas entre nodos y enlaces + 	**Se utiliza para operaciones de clase más concisas.** A partir de requerimientos y datos de entrada. El objetivo es verificar que el software cumpla con los requisitos ejecutando el código. No analiza el código por dentro Ignora estructuras de control. Responden a preguntas como: ➔ ¿Es sensible el sistema a ciertos valores de entrada? ➔ ¿Qué volúmenes y niveles de datos tolerará el sistema? ➔ ¿Qué efectos sobre las operaciones del sistema tendrán determinadas combinaciones de datos? Intentan encontrar: ● Funciones u operaciones incorrectas o ausentes ● Errores de interfaz ● Errores en estructuras de datos o de acceso a base de datos

1 global.

¿Cómo se calcula? 3 opciones:

1. Sumar regiones
2. Enlaces - Nodos + 2
3. Nodos predicados + 1

Número de casos de prueba = 3

Casos de prueba

A – B – D - G – I
A – B – E – G – I
A – C – F – H – I

- **Prueba de condiciones:** prueba de las estructuras de control.
  - Ejercita las condiciones lógicas
  - Se comprueban las ramas V y F de cada condición
  - Se cubran cada una de las restricciones de cada condición.

  Se realiza para condiciones válidas esperadas e inesperadas.

Errores típicos:

- Un operador lógico (y/o equivocados)
- Variable de control
- Operador relacional
- Expresión aritmética.

- **Prueba de bucles o ciclos:** debe probarse la no existencia de bucles infinitos. Casos de prueba: ciclos con cuentas, búsqueda de un valor mayor, considerando o no el cero y el límite del vector.



- **Prueba de flujo de datos:**
  - Seleccionar caminos de prueba en base al lugar donde se definen y se usan las variables.
  - Seguimiento de variables.
  - Partes que necesitan mayor análisis.

Características

No ejecuta el SW sino que lo analiza por dentro

Tiende a garantizar que:

- Se ejerciten al menos una vez todos los caminos independientes de cada módulo.
- Se analicen las decisiones lógicas en sus partes verdadera y falsa.
- Se ejecutan todos los ciclos en sus límites.
- Se verifican las estructuras internas de datos asegurando su validez.

- Fallas de rendimiento
- Problemas de inicio o fin

Técnicas de pruebas de caja negra

- **Partición equivalente**

Descubrir clases de errores → Reducir cantidad de pruebas

Clases de equivalencia válidas → entradas validas o invalidas

- **Análisis de frontera o de valores límites:** Ejercitación de valores en los límites: rangos y vectores.
- **Prueba basada en modelo (Programación estructurada y POO):** A partir del diagrama de estado, se ve su validez.
- **Prueba de aplicaciones orientadas a objetos**
  - Prueba basada en el escenario (Caso de uso) del usuario
  - Prueba basada en comportamiento (Diagrama de clases)
  - Prueba basada en fallos: puede encontrar muchos errores con poco esfuerzo.
  - Prueba aleatoria: al azar.
  - Prueba de partición → Categorías:
    - Partición en el nivel de clase (entrada y salida)
    - Partición con base en estado (Cambio o no de estado)
    - Partición en base a categorías (Por función).

Depuración/Debbuging: Proceso destinado a solucionar un error encontrado y su causa. Elimina un error hallado, pero no detecta el error ni es una prueba. Existen tres técnicas de depuración:

1. Fuerza bruta: + común y - eficiente, cuando todo lo demás falla.
2. Vuelta atrás: en programas pequeños, se rastrea hacia atrás hasta encontrar la causa.
3. Eliminación de causas: mediante la inducción o deducción, se plantean causas potenciales y se prueban casos para refutar la hipótesis.

CLASES PRÁCTICAS

18.03.2024

- ➔ El diseño depende de la construcción que queramos.
- ➔ En el centro del sistema están los datos y alrededor están las pantallas. Cada pantalla hace una sola operación.
- ➔ Reglas de diseño:
  - ◆ Responsabilidad única: cada elemento tiene una responsabilidad, hace una cosa.
  - ◆ Mejor disperso que denso → ante la duda, separar. Es mejor tener muchas pantallas fáciles que una enorme.
  - ◆ Todo tiene estado, hasta que se pruebe lo contrario. Ante la duda, siempre poner un estado.
  - ◆ Uno tiene que tener acceso a los datos que derivan de tu responsabilidad y no más.

    - Manejo del cambio → el código se escribe desde el primer día pero se sigue modificando por años.
- ➔ La pantalla habla con los datos de ida y de vuelta.

21.03.2024
Control por posición de interés: Segregación de funciones para el control interno de la compañía y mejorar la información y la calidad de atención.

4 niveles: Global, de dominio, componentes y a detalle

- **Nivel Global:** En este nivel, se establecen los objetivos generales del sistema y se definen los requisitos de alto nivel. Es un nivel de abstracción muy alto y se centra en los aspectos más generales y amplios del sistema. Aquí se definen las funciones principales del sistema, su propósito general, los stakeholders involucrados y las restricciones principales. Se utilizan técnicas como el modelado de negocio, la definición de casos de uso y la creación de diagramas de contexto para comprender la interacción del sistema con su entorno.
- **Nivel de Dominio**: En este nivel, se profundiza en los aspectos específicos del dominio del problema que el sistema aborda. Se identifican y definen los conceptos clave, las reglas de negocio y las relaciones entre entidades del dominio. Este nivel se centra en comprender a fondo el problema que el sistema está diseñado para resolver y en traducir ese entendimiento en un modelo conceptual. Se utilizan técnicas como el modelado de dominio, la creación de diagramas de clases y la especificación de requisitos funcionales y no funcionales.
- **Nivel de Componentes:** Aquí se aborda el diseño y la especificación de los componentes individuales que componen el sistema. Los componentes pueden ser módulos, clases, servicios o cualquier otra unidad de construcción del sistema. En este nivel, se definen las interfaces entre los componentes, sus responsabilidades y las interacciones entre ellos. Se busca lograr un diseño modular y cohesivo que facilite la reutilización, la mantenibilidad y la escalabilidad del sistema. Se utilizan técnicas como el diseño orientado a objetos, la especificación de interfaces y la creación de diagramas de componentes y de despliegue.
- **Nivel a Detalle**: Este nivel se enfoca en la implementación concreta de cada componente del sistema. Se escriben el código fuente, las consultas de base de datos, los scripts de configuración y cualquier otro artefacto necesario para construir el sistema. En este nivel, se tienen en cuenta consideraciones técnicas específicas, como el rendimiento, la seguridad y la eficiencia. También se realizan pruebas unitarias y de integración para verificar que cada componente funcione correctamente y cumpla con los requisitos establecidos. Este nivel implica un alto grado de detalle y es donde se concretan las decisiones tomadas en los niveles anteriores.

Estos cuatro niveles de abstracción permiten descomponer un sistema complejo en partes más manejables y comprensibles, lo que facilita su diseño, desarrollo y mantenimiento a lo largo del ciclo de vida del software.

El esta software compuesto por **datos y programas**
Construimos gestores de datos.

- **documentación de desarrollo** que permite justificar las decisiones y prepararse para el mantenimiento. Sirve para ordenar, justificar recursos y ver qué decisiones tomé para hacer el mantenimiento.
- **documentación de usuario** es el manual de usuario(herramienta que necesita el usuario para trabajar con el sw sin capacitación). En esta documentación se documenta un ejemplo de la manera más simple.

Esto permite que el software se puede construir y mantener
Debemos ser disciplinados y llevar a cabo buenas prácticas para tener un <u>buen estándar de desarrollo.</u>

- <u>Programas a medida:</u> Diseñados específicamente para un cliente, adaptados a sus necesidades únicas y construidos desde cero según sus requisitos. Requieren un proceso de desarrollo personalizado y un equipo dedicado para su implementación.
- <u>Programas estandarizados/enlatados:</u> Son soluciones listas para usar, disponibles comercialmente para un público amplio. No son personalizables como los programas a medida, pero ofrecen funcionalidades

generales y son accesibles de inmediato. Ejemplos incluyen suites de productividad como Microsoft Office y software de edición de imágenes como Adobe Photoshop.

Los software van deteriorándose por:
- Errores y actualizaciones
- Obsolescencia tecnológica
- Nuevos casos que denominamos raros para procesar.
- Seguridad

Tengo que tener redundancia de sistemas para aumentar la fiabilidad mediante duplicación. Por ejemplo, tener un juego de llaves extra para tu casa.

Software debe ser fiable → realiza consistentemente sus funciones de manera precisa y predecible, sin errores o fallos inesperados

Es importante entender el tiempo del usuario frente a la aplicación para entender cuál es su tarea y el tiempo de su tarea. Si su tarea es comprobación debemos garantizar que el usuario no tenga que ir y volver.

La usabilidad: La aplicación debe ser parte de la tarea como una ayuda pero el usuario es quien la gestiona. Se trata de diseñar productos y sistemas que sean fáciles de usar y que proporcionen una experiencia positiva para el usuario, lo que a su vez puede mejorar la eficacia, la productividad y la satisfacción del usuario.

Fácil de mantener:
- estándares
- Facilidad de acceso
- Buen diseño
    - Modularización
    - Encapsulamiento del código en clases para facilitar el mantenimiento
- Versionado de código
- Entregas frecuentes: Maneja la ansiedad del cliente y de los usuarios. Por otro lado es importante tener en cuenta que el software puede desbalancear los intereses de los usuarios.Quienes están a favor del cambio quieren todo para ya y los que no para más adelante.

La programación no es rutinaria y hay permanentes tomas de decisiones.

Repaso: Principio de responsabilidad única. Mejor muchas pantallas simples que pocas pero complicadas.
Ej: Cargar factura: se puede dividir. No evalúan mucho la usabilidad.
Las pantallas hablan con la base de datos, no entre ellas. No son las pantallas las que tienen secuencias, son los datos los que manejan secuencias (estados).
Orden coherente de los estados forzando al usuario a seguir una secuencia.

Programar pensando en objetos. Un objeto en c# es la representación en código de un objeto en la vida real.
Ese objeto puede volver modificado, como no.
Cuanto más prolijos seamos definiendo lo que se puede hacer y no con un objeto, menos alteraciones recibirá. Se puede tener mecanismos que protejan a los objetos de ser modificados.

En .net TODO es un objeto.
Palabras clave en .net: If, (), {}, get, set, public, class
Todo lo que no son palabras claves, son objetos.
Variables: No es un valor, no tiene un valor. Es solamente una especie de “flecha” que apunta a un objeto.
Creación de un nuevo objeto → ***Factura a = new Factura ();*** “a” no es nada(→). La factura es una instancia de una clase. Los objetos existen aunque no haya variables (flechas). Que la flecha apunte siempre al mismo tipo de objeto. No se pueden guardar los objetos.

Un objeto se crea cuando existe la palabra ***new.***

- Si hago ***Factura b = a;***

b apunta a lo mismo que apunta *a*. Primero se resuelve la parte de la derecha (a) y llega a un objeto. A ese mismo objeto debe apunta b

- Si hago ***a = null;***

null es una referencia vacía. Cuando vos haces esto, *a* pasaría a apuntar a la nada.

Object:

1. Value Types: string, int, long, single, bool, datetime, decimal (son inmutables, no puede cambiar sin dejar de ser el mismo)
2. Reference Type: lista, array, textbox.

El formulario tiene un único objetivo → capturar y validar los datos del usuario.
La clase es la que controla el formulario.
Domain → tiene operaciones que no están en la pantalla. Solo tiene dos salidas: OK o ERROR DE VALIDACIÓN (esperable porque es un error por culpa de los datos del usuario).
Metáfora cáscara de huevo con los errores.
Hasta que no apreto "aceptar", no pasa nada

Formulario

- ❖ Es una clase como cualquier otra
- ❖ Se genera a partir de los objetos visuales que usamos
- ❖ El objetivo de esta clase es manejar/hacer que funcione el forms
- ❖ No se ocupa de conseguir los datos ni saber de donde salen
- ❖ Es la clase "directora": maneja lo que va pasando
- ❖ Aca se establece qué pasa cuando se toca un botón, cuando se escribe un texto,etc.
- ❖ Contiene listas, botones, textbox, checkbox
- ❖ Un archivo por clase

Modelo

- ❖ Esta clase agrupa, consigue y pone disponible los datos
- ❖ Solo sabe de datos (total de ventas, facturas, etc). No sabe lo que es un botón, un textbox, etc.
- ❖ La mayor parte del código se escribe aca.

Lo primero que ocurre es el **Load:** Evento que marca la carga del formulario. Permite que el usuario lo vea.
Va a tener una variable que sea modelo y lo primero que hará el Load será: Modelo = New Modelo
Apenas me cargo, necesito que alguien me dé los datos.
Esta separación entre el forms y el modelo sirve para separar la parte visual de la de los datos.
Por ejemplo si en el formulario creo una lista con clientes, productos y stock, vamos a necesitar todos esos datos que nos lo va a dar el modelo. Este va a crear una clase itemStock que va a tener una clase cliente, una producto y otra de stock. Con eso crea un array.

Debe mostrar los datos de manera simple y elemental en el formulario.
Debe dejarla servida a la pantalla. (La pantalla no combina datos, no pasa enteros a string) El modelo construye los datos para la pantalla.

Los DFD contienen los módulos y archivos. Los subsistemas del DFD son módulos que vamos a necesitar crear
Dentro de cada modelos están los siguientes componentes:

- Módulos (Subsistemas):
  - EJ: Clientes (ClienteModulo), Proveedores (ProveedorModulo).
  - Todas las modificaciones de datos las debe hacer un módulo. El modelo ESCRIBE sobre los datos.

    - Realizan las operaciones
- Archivos
    - Ej: Facturas (FacturaArchivo), Usuarios (UsuarioArchivo), Total de ventas (TotalVentaArchivo).
    - Son la representación de los datos.
    - Funcionan como tablas. Ej: el archivo Usuarios tiene columnas que representan sus propiedades y las filas son cada uno de los usuarios.
    - Los modelos van a necesitar recurrir a estos archivos para obtener los datos. El modelo LEE los datos.
    - Cada archivo contiene un .json por dato: Factura1.json. Factura2.json, etc.
    - Puedo extraer filas enteras

CADA PANTALLA TIENE SU MODELO. Nunca se comparte un modelo entre 2 pantallas

Nombre de clase (Nomenclatura determinada): VentasForm

1. De qué trata (ej:Ventas)
2. De qué es (ej:Form)

Creo una carpeta que sea Ventas que va a contener a VentasForm, VentasModelo (y sus subclases)
El código no es un método de seguridad. No cambia si es Public o Private.

Pasos para crear nuestro sistema:

1) **Diseñar la pantalla** -armo prototipo visual y luego en visual studio
    a) Acomodar textos, Colores, Listas, Nombre a las cosas
2) **Punto de control**: Imaginamos la interacción de la pantalla según el enunciado.
3) **Punto de acción**: Invento archivos para hacer la consulta (Prototipo)
    a) Corroboro qué otras clases necesito.
    b) Ver qué interacción con la pantalla funciona.
    c) Permite que el usuario valide el prototipo.
    d) Datos similares a los del cliente.
4) **Punto de acción:** Trabajar internamente. Qué método trabaja con qué lista. Ya sé qué viaja entre el formulario y el modelo.
5) **Punto de acción:** ArchivoCliente - Tengo una clase que se llama cliente que viaja entre el modelo y los archivos. y entre los modelos y los módulos. Siempre viajan estos modelos completos.
    a) Modelo de Alta de factura El archivo recibe la factura completa. No recibe una agrupación de datos. Recibe un objeto completo de factura.
    b) La pantalla es la que valida los datos y sus formatos.
    c) Modelo valida que en conjunto estos datos tengan sentido.
    d) Se valida consistencia interna del cliente.
    e) El módulo valida los datos de escritura y no de lectura. Validar datos de negocio. Tienen que ser válidas antes de hacerla. Ej: Si el cliente ya existe.

Hay 3 niveles de validación:

1. Que el tipo de dato sea correcto. Esto lo hace el formulario
2. Que el conjunto de datos que ingresó el usuario tenga sentido. Esto lo hace el modelo
3. Que lo que se ingresa esté correcto de acuerdo a los datos del sistema (validación de negocio). Esto lo hace el módulo.

Los datos de los archivos están bien, los datos no se validan.

Al diseñar el DFD, realizamos globos y flechas de E/S. En las flechas viajan los .json. Cuando diseño módulos y los globos, entran y salen proveedores del módulo de proveedores. Viajan preguntas y respuestas.

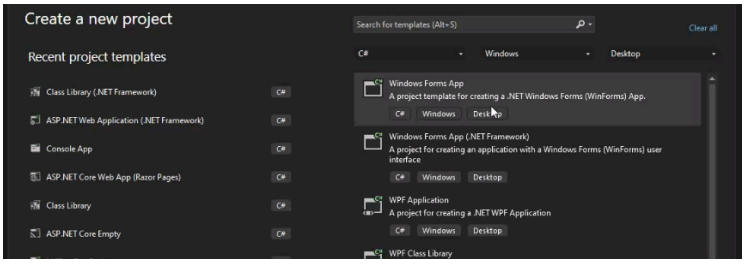


Si vos queres solo proveedores activos que pasen del proveedor al globo, debe haber previamente una entrada donde se establezca el criterio de la salida esperada.
Un globo solo habla con **archivos.**
Un globo representa 1 o más pantallas y módulos:



Clase práctica 1 (18/04)
Crear nuevo proyecto >> Usar Windows Forms App
Nombre del proyecto: Nombre propio >> Ejercicio >> N°
Ejemplo: AndresPanish.Prototipos.EJ01
Reglas:

- Un ejercicio, un proyecto, un repositorio
- Cada archivo debe tener una sola clase → El nombre del archivo tiene que ser el nombre de la clase
- Cada formulario debe tener solo un modelo.
- Primero valido todo y después opero.



PANTALLA: Formulario en que el usuario completa datos y el sistema valida.
El nombre del formulario se debe cambiar ni bien se empieza para evitar errores. EJ: Ejercicio01Form.cs
El formulario no debe hacer nada hasta que el usuario no le dé al botón de aceptar. Recién ahí se hacen todas las validaciones, operaciones, etc.

```
1 reference
public Ejercicio01Form()
{
    InitializeComponent();
}
```

Para ver el código detrás de lo visual → F12 .Designer
El método constructor de la clase de un formulario es un método que se ejecuta cada vez que se crea un elemento nuevo de esta clase.

Cada herramienta es un objeto. A cada objeto en el forms hay que asignarle un nombre (tipo de botón + que representa) Ej: txtNombre, txtFechaNacimiento, btnAceptar.
1° lo que soy 2° lo que hago. Ejemplo: BotónAceptar → ser consistentes

Tenemos que tener una variable a nivel de formulario para tener una referencia al modelo de ese formulario. Ejercicio01Model modelo = new Ejercicio01Model();

```
3 references
public partial class Ejercicio01Form : Form
{
    private readonly Ejercicio01Model modelo = new Ejercicio01Model();

    1 reference
    public Ejercicio01Form()
    {
        InitializeComponent();
    }

    1 reference
    private void Ejercicio01Form_Load(object sender, EventArgs e)
    {

    }

    1 reference
    private void BotonAceptar_Click(object sender, EventArgs e)
    {
        var persona = new Persona();
        persona.Dni = this.TxtDocumento.Text;
        persona.Apellido = this.TxtApellido.Text;
        persona.Nombre = this.TxtNombre.Text;
        persona.FechaNacimiento = this.TxtFechaNacimiento.Text;

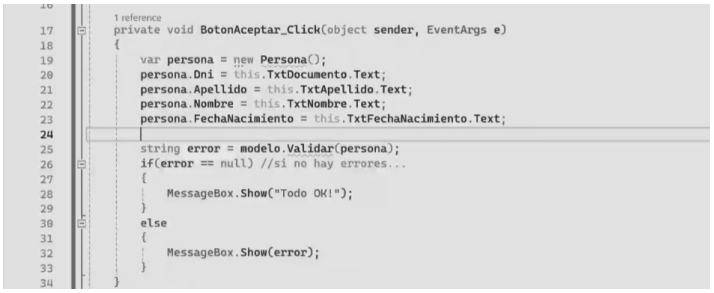
        modelo.Validar(persona);
    }
}
```

Todas las acciones que se tienen que ejecutar antes que el formulario se ponen en el método Load
El formulario solo tiene que hacer validaciones de tipo de dato.

El objetivo de un formulario es armar un objeto con los datos que tiene y pasarselo al modelo para que los valide.
El modelo es una frontera entre la lógica de datos y la pantalla.
Tenemos que empezar codificando a alto nivel. Primero definimos todas las clases y después tenemos que empezar a definir las validaciones de cada una.

En el DFD2 ya sabemos cuales son las pantallas. Algunos globitos son pantallas y otros son módulos. Pantallas de ventas, log in, para ingresar una orden, para completar una orden, etc.
La idea es que el proyecto siga al diseño y que el diseño siga al proyecto.

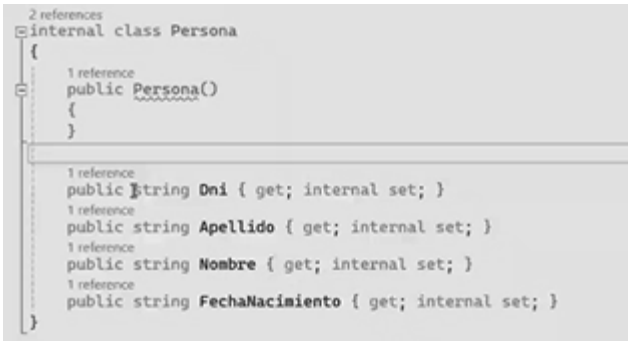
Cuando nos aparece una parte del código subrayada en rojo (como el caso del new Persona() de abajo) podemos tocar ctrl + punto para que nos lo solucione. En este caso había que crear la clase persona y sus propiedades.



```
1 reference
private void BotonAceptar_Click(object sender, EventArgs e)
{
    var persona = new Persona();
    persona.Dni = this.TxtDocumento.Text;
    persona.Apellido = this.TxtApellido.Text;
    persona.Nombre = this.TxtNombre.Text;
    persona.FechaNacimiento = this.TxtFechaNacimiento.Text;

    string error = modelo.Validar(persona);
    if(error == null) //si no hay errores...
    {
        MessageBox.Show("Todo OK!");
    }
    else
    {
        MessageBox.Show(error);
    }
}
```

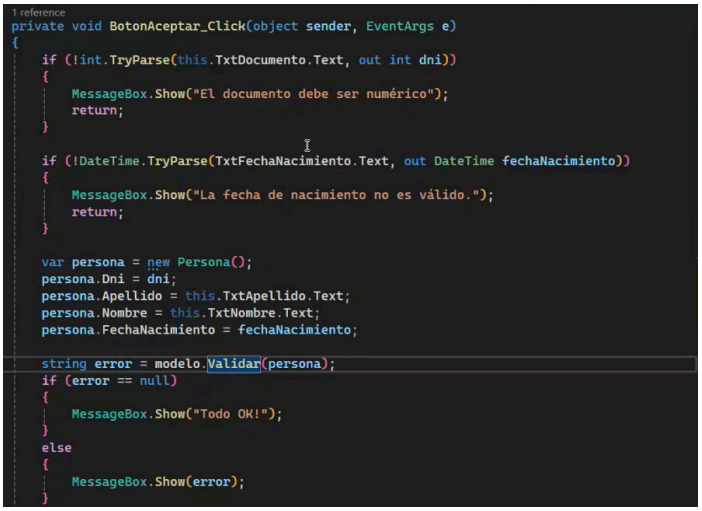
Debemos crear la nueva clase Persona y sus propiedades/datos. Luego hay que ver que tipo de datos tienen estas propiedades.
Los tipos de dato de por ejemplo, número de orden debemos preguntarle al cliente.



```
2 references
internal class Persona
{
    1 reference
    public Persona()
    {
    }

    1 reference
    public string Dni { get; internal set; }
    1 reference
    public string Apellido { get; internal set; }
    1 reference
    public string Nombre { get; internal set; }
    1 reference
    public string FechaNacimiento { get; internal set; }
}
```

El formulario tiene que hacer validaciones de tipo de dato. Las mínimas e indispensables validaciones para armar el objeto y pasarselo al método.
Para realizar validaciones de tipo realizamos lo siguiente:



```
1 reference
private void BotonAceptar_Click(object sender, EventArgs e)
{
    if (!int.TryParse(this.TxtDocumento.Text, out int dni))
    {
        MessageBox.Show("El documento debe ser numérico");
        return;
    }

    if (!DateTime.TryParse(TxtFechaNacimiento.Text, out DateTime fechaNacimiento))
    {
        MessageBox.Show("La fecha de nacimiento no es válido.");
        return;
    }

    var persona = new Persona();
    persona.Dni = dni;
    persona.Apellido = this.TxtApellido.Text;
    persona.Nombre = this.TxtNombre.Text;
    persona.FechaNacimiento = fechaNacimiento;

    string error = modelo.Validar(persona);
    if (error == null)
    {
        MessageBox.Show("Todo OK!");
    }
    else
    {
        MessageBox.Show(error);
    }
```

El objetivo del método “Validar” es determinar si la clase (en este caso Persona) está bien o mal y devolver un mensaje de error en caso de una falla.

Se puede sumar un resumen de qué hace el método

La segunda validación las hace el objeto con sus propios datos(dentro del modelo):

```
internal string Validar()
{
    if (Dni < 1_000_000 || Dni > 99_999_999)
    {
        return "El DNI debe tener entre 7 y 8 dígitos.";
    }

    if (string.IsNullOrWhiteSpace(Apellido))
    {
        return "El apellido es requerido.";
    }

    if (Apellido.Length > 50)
    {
        return "El apellido debe tener menos de 50 caracteres.";
    }


    if (string.IsNullOrWhiteSpace(Nombre))
    {
        return "El nombre es requerido.";
    }

    if (Nombre.Length > 50)
    {
        return "El nombre debe tener menos de 50 caracteres.";
    }
```

Clase práctica 2 (25/4)

PANTALLA: ABM (Altas, bajas y modificaciones) - Formulario en que un usuario puede dar de alta/baja/modificar (operaciones básicas) una lista plana de objetos. Tenemos un archivo con ciertos datos tabulares (con filas y columnas) y queremos editarlos con una pantalla.

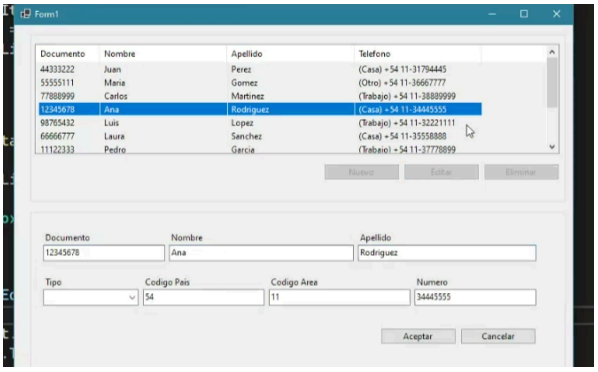
Cada registro es una serie de propiedades/datos, ninguno es una lista.
Cada formulario tiene su propia clase Modelo.

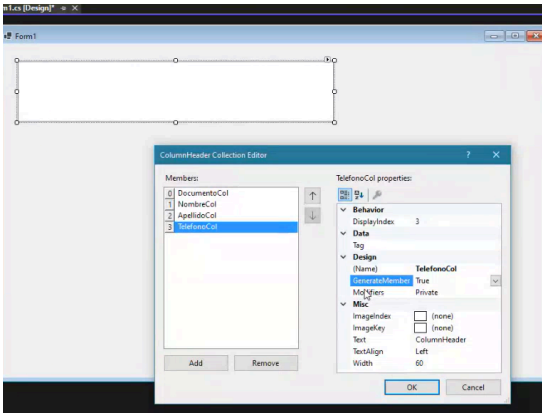
```
namespace AndresPa.Prototipos.Ejercicio04;

3 references
public partial class PersonasForm : Form
{
    PersonasModelo modelo = new();

    1 reference
    public PersonasForm()
    {
        InitializeComponent();
    }
}
```

En nuestro ejemplo tenemos una lista Persona con las propiedades de Documento, Nombre, Apellido y Telefono en la que ninguna de estas propiedades es una lista en sí misma.
El teléfono está compuesto del código de área y el número de teléfono. Hay que concatenarlos. Teléfono va a ser un objeto en sí mismo.





GridView: Sirve para mostrar los datos en una tabla (donde cada columna representa un campo y cada fila representa un registro). Es difícil configurarlo bien, es muy profesional → Usamos ListView.
Con la propiedad FormBorderStyle podemos cambiar el tamaño del formulario
La lista es un objeto y cada columna de la lista también es un objeto
Para agregar columnas hay que darle al click derecho en la lista.

```
1 reference
private void PersonasForm_Load(object sender, EventArgs e)
{
    modelo = new();

    TipoCombo.Items.Add("Casa");
    TipoCombo.Items.Add("Trabajo");
    TipoCombo.Items.Add("Otro");

    foreach (var persona in modelo.Personas)
    {
        var fila = new ListViewItem();
        fila.Text = persona.Documento.ToString();
        fila.SubItems.Add(persona.Nombre);
        fila.SubItems.Add(persona.Apellido);
        fila.SubItems.Add(persona.Telefono.Descripcion());
        fila.Tag = persona;
        PersonasList.Items.Add(fila);
    }
```

Luego de agregar las columnas debemos establecer que para cada persona en el modelo se tiene que cargar en la lista. La lista de personas tiene una propiedad “Item” que le agrega un objeto llamado ListViewItem que representa un objeto en la lista. Con la línea var fila = new ListViewItem (); indica que va hacer algo con cada fila de lista.

Luego creamos la propiedad Personas en el modelo como una lista de objetos persona:
public List<Persona> Personas { get; set; }

```
1   namespace AndresPa.Prototipos.Ejercicio04;
2
    2 references
3   internal class PersonasModelo
4   {
5       public List<Persona> Personas { get; set; }
6
7   }
```

Y luego se crea la clase Persona y Teléfono

```
internal class Persona
{
    1 reference
    public int Documento { get; set; }
    1 reference
    public string Nombre { get; set; }
    1 reference
    public string Apellido { get; set; }
    1 reference
    public Telefono Telefono { get; set; }
}
```

```
internal class Telefono
{
    0 references
    public TipoTelefono Tipo { get; set; }
    1 reference
    public int CodPais { get; set; }
    1 reference
    public int CodArea { get; set; }
    1 reference
    public int Numero { get; set; }

    1 reference
    public string Descripcion()
    {
        return $"+{CodPais:00} {CodArea:00}-{Numero:0000}";
    }
}
```

*Lo que va entre llaves son las variables y lo de después de los “:” es el formato de esa variable.

Después tenemos que crear un constructor en el modelo PersonasModelo() que lo vamos a usar para darle datos de prueba y saber si funciona la pantalla, para después conectarla al resto del sistema.

```
public PersonasModelo()
{
    Personas = new List<Persona>
    {
        new Persona { Apellido = "Perez", Nombre = "Juan", Documento = 45634987, Telefono = new Telefono { Tipo = "Casa", CodPais = 54,
        new Persona { Apellido = "Gomez", Nombre = "Maria", Documento = 12345678, Telefono = new Telefono { Tipo = "Móvil", CodPais = 5
        new Persona { Apellido = "Lopez", Nombre = "Carlos", Documento = 87654321, Telefono = new Telefono { Tipo = "Oficina", CodPais =
        new Persona { Apellido = "Martinez", Nombre = "Lucia", Documento = 23456789, Telefono = new Telefono { Tipo = "Casa", CodPais =
        new Persona { Apellido = "Fernandez", Nombre = "Miguel", Documento = 98765432, Telefono = new Telefono { Tipo = "Móvil", CodPais
    };
}
```

En la pantalla, tendremos una lista de personas.

1. Con Editar, edito un contacto y se habilita el GroupBox

2. Con Eliminar, se elimina contacto de la lista seleccionada

```
foreach(var persona in modelo.Personas)
{
    var fila = new ListViewItem();
    fila.Text = persona.Documento.ToString();
    fila.SubItems.Add(persona.Nombre);
    fila.SubItems.Add(persona.Apellido);
    fila.SubItems.Add(persona.Telefono.Descripcion());
    fila.Tag = persona;
    PersonasList.Items.Add(fila);
}
```

Le sumamos esta propiedad tag voy a guardarme una referencia a la persona correspondiente. El tag es propiedad object porque debe poder contener a cualquier objeto.

➔ **Botón Editar:** La persona debe seleccionar una fila. Una vez hecho esto el sistema saca del tag a la persona a editar. Cuando lo saco, hay que especificar a qué objeto hace referencia (en este caso a Persona). Se pasa de una referencia de tipo objeto a una de tipo persona. Esto se conoce como castear (se usa cuando tengo una referencia general y quiero tener una más específica).
Ej: Tengo la referencia "documento" y lo casteo a una referencia "factura". Luego se pasan los datos del objeto al group para que pueda editarlos. Inhabilita la edición desde la lista y habilita la edición desde el group. En el modelo tenemos que agregar el objeto PersonaEnEdicion.

```
private void EditarBoton_Click(object sender, EventArgs e)
{
    if (PersonasList.SelectedItems.Count == 0)
    {
        MessageBox.Show("Seleccione un item de la lista primero.");
        return;
    }

    var personaAEditar = (Persona)PersonasList.SelectedItems[0].Tag;

    DocumentoText.Text = personaAEditar.Documento.ToString();
    ApellidoText.Text = personaAEditar.Apellido;
    NombreText.Text = personaAEditar.Nombre;
    TipoCombo.SelectedItem = personaAEditar.Telefono.Tipo.ToString();
    CodAreaText.Text = personaAEditar.Telefono.CodArea.ToString();
    CodPaisText.Text = personaAEditar.Telefono.CodPais.ToString();
    NumeroText.Text = personaAEditar.Telefono.Numero.ToString();

    ListaGroup.Enabled = false;
    EdicionGroup.Enabled = true;

    modelo.PersonaEnEdicion = personaAEditar;
}
```

➔ **Botón Cancelar:** Se vacían todos los campos y se vuelve a habilitar la lista y a inhabilitar la edición.

```
1 reference
private void CancelarBoton_Click(object sender, EventArgs e)
{
    DocumentoText.Text = string.Empty;
    ApellidoText.Text = string.Empty;
    NombreText.Text = string.Empty;

    TipoCombo.SelectedItem = string.Empty;
    CodAreaText.Text = string.Empty;
    CodPaisText.Text = string.Empty;
    NumeroText.Text = string.Empty;

    ListaGroup.Enabled = true;
    EdicionGroup.Enabled = false;
}
```

➔ **Botón Nuevo:** Habilita la parte de edición pero sin cargar nada

```
1 reference
private void NuevoBoton_Click(object sender, EventArgs e)
{
    ListaGroup.Enabled = false;
    EdicionGroup.Enabled = true;
    modelo.PersonaEnEdicion = null;
}
```

➔ **Botón Aceptar:** Agrega los datos que la persona editó al modelo y valida los datos.

```
private void AceptarBoton_Click(object sender, EventArgs e)
{
    //TODO: validar.
    if (!int.TryParse(DocumentoText.Text, out var doc))
    {
        MessageBox.Show("El documento no es válido");
        return;
    }

    //De todas maneras, es un combo asi que no hacía falta.
    /*
    if(!Enum.TryParse<TipoTelefono>(TipoCombo.Text, out var tipoT
    {
        MessageBox.Show("El tipo de telefono no es válido");
        return;
    }
    */

    //etc....
```

La única responsabilidad de la pantalla es armar un “paquete” de datos y pasarselo al modelo, no modifica los datos. La pantalla no modifica a la persona ni tiene que saberlo, es el modelo el que se encarga de esto.
Este es el paquete que le pasamos al modelo(va dentro del AceptarBoton_Click)

```
Persona persona = new()
{
    Documento = doc,
    Apellido = ApellidoText.Text,
    Nombre = NombreText.Text,
    Telefono = new Telefono
    {
        Tipo = Enum.Parse<TipoTelefono>(TipoCombo.Text),
        CodPais = int.Parse(CodPaisText.Text),
        CodArea = int.Parse(CodAreaText.Text),
        Numero = int.Parse(NumeroText.Text)
    }
};

string error = modelo.Modificar(persona);
if (error != null)
{
    MessageBox.Show(error);
    return;
}

CargarLista();
```

Luego que se modifica una persona, se tiene que recargar la lista. Creamos el método de cargar lista

```
1 reference
private void CargarLista()
{
    PersonasList.Items.Clear();

    foreach (var persona in modelo.Personas)
    {
        var fila = new ListViewItem();
        fila.Text = persona.Documento.ToString();
        fila.SubItems.Add(persona.Nombre);
        fila.SubItems.Add(persona.Apellido);
        fila.SubItems.Add(persona.Telefono.Descripcion());
        fila.Tag = persona;
        PersonasList.Items.Add(fila);
    }
}
```

El proceso del formulario comienza creándose el modelo, se hacen los arreglos a los controles y se carga la lista. Luego, cuando se le da al botón aceptar se hacen las validaciones mínimas, se crea el paquete de datos y se lo pasa al modelo. Si encuentra un error lo muestra y si no se hace la modificación. Se borran los items de la lista y se recarga la lista y ahí termina.
Tenemos que crear el procedimiento de Modificar en el modelo. Queremos que la persona haga todas las validaciones que pueda por sí misma.

```
1 reference
public string Modificar(Persona datosIngresadosPersona)
{
    string error = datosIngresadosPersona.Validar();
    if (error != null)
    {
        return error;
    }

    //TODO: validaciones que se realizan tanto para una modificacion
    //como

    if (PersonaEnEdicion == null) //estoy dando de alta.
    {
        //Ej.: no quiero 2 personas con el mismo.
        foreach (var personaExistente in Personas)
        {
            if (personaExistente.Documento == datosIngresadosPersona.Documento)
            {
                return "Ya existe una persona con el documento indicado.";
            }
        }
    }
    else //estoy modificando.
    {
        //Ej.: NO se puede modificar el nro de documento.
        if (PersonaEnEdicion.Documento != datosIngresadosPersona.Documento)
        {
            return "No puede modificar el documento.";
        }

        //Posibilidad 1: reemplazar un objeto con otro.
        //OJOOO: No siempre es valido hacer esto.
        //Personas.Remove(PersonaEnEdicion);
        //Personas.Add(datosIngresadosPersona);

        //Siempre es valido "pisar" los datos sin cambiar el objeto.
        PersonaEnEdicion.ActualizarCon(datosIngresadosPersona);
    }
```

Las validaciones que van aca son las validaciones de negocio. Por ejemplo: Cuando doy de alta una persona no queremos que haya dos personas con el mismo documento. O si estoy editando no se debe poder modificar el documento.
Lo mejor es decirle al objeto que se actualice por si misma. Lo ponemos dentro de la clase Persona.
Todo esto lo hacemos por si actualizo el teléfono, el nombre,etc no repercute en otras partes del código. Queda todo encapsulado.

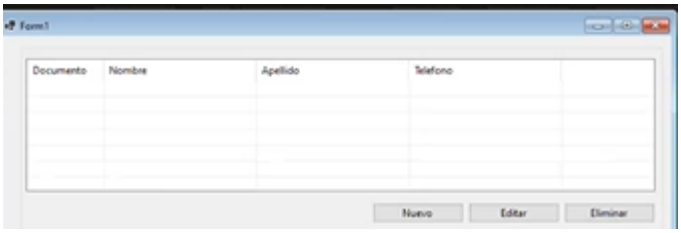
```
18 references
internal class Persona
{
    19 references
    public int Documento { get; set; }
    15 references
    public string Nombre { get; set; }
    15 references
    public string Apellido { get; set; }
    18 references
    public Telefono Telefono { get; set; }

    1 reference
    public void ActualizarCon(Persona datos)
    {
        Documento = datos.Documento;
        Nombre = datos.Nombre;
        Apellido = datos.Apellido;
        Telefono.ActualizarCon(datos.Telefono);
    }
}
```

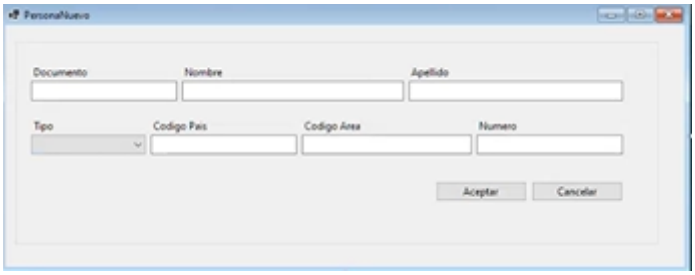
Cuando una propiedad puede ser nula hay que agregarle un "?" después de poner el nombre de la propiedad. Ej: public Persona? PersonaEnEdicion { get; set; }.

Clase práctica 3 (02/05) - Continuación del ej.4 de la guía
Cuando el formulario se vuelve muy complejo, no sirve hacer una sola pantalla con todas las funcionalidades, es mejor dividir. Hay que pensar la lista como un menú, y no tanto como una lista. Tenemos que hacer que el usuario navegue por un formulario a la vez, en lo posible evitar dos formularios en pantalla funcionando al mismo tiempo. Para que funcione, todos los formularios tienen que compartir un mismo objeto modelo. Voy a partir el formulario en 2 distintos.
Una operación es una vista que se puede dividir en varios formularios (siempre y cuando compartan el mismo modelo). Tiene que funcionar un formulario a la vez.

1. **Formulario principal:** Lista con sus operaciones



2. **Un formulario para cada una de las operaciones que se pueden hacer (Nuevo/Editar/Eliminar):**



Pasos a seguir:
Creo un nuevo formulario (puede ser el nombre de la operación, PersonaNuevoForm, por ejemplo)
El formulario PersonaNuevoForm ya no crea su propio modelo, lo recibe. Hay que convertir a la clase PersonasModelo en una clase pública

En el *formulario principal* el botón "nuevo" va a crear un nuevo formulario. Ademas le tenemos que compartir el modelo al formulario que creamos: FormNuevo.Modelo = modelo;. Con esto vamos a poder ver los cambios que se hagan en el otro formulario.
Agrego la función ShowDialog para que no se puedan usar 2 formularios a la vez. La lista principal funciona como un menú que muestra la lista y lo único que sabe es derivar al usuario a cada una de las operaciones.
El formulario principal solo borra la lista y la actualiza. No sabe qué más pasa
Quedaría algo así:

```
1 reference
private void NuevoBoton_Click(object sender, EventArgs e)
{
    var formNuevo = new PersonaNuevoForm();
    formNuevo.Modelo = modelo;
    formNuevo.ShowDialog(); //La ejecución se detiene en esta línea....

    //.... y continúa sólo una vez que el formulario se haya cerrado.
    CargarLista();
}

3 references
private void CargarLista()
{
    PersonasList.Items.Clear();

    foreach (var persona in modelo.Personas)
    {
        var fila = new ListViewItem();
        fila.Text = persona.Documento.ToString();
        fila.SubItems.Add(persona.Nombre);
        fila.SubItems.Add(persona.Apellido);
        fila.SubItems.Add(persona.Telefono.Descripcion());
        fila.Tag = persona;
        PersonasList.Items.Add(fila);
    }
```

Y en el formulario *PersonaNuevoForm* quedarían así los botones aceptar/cancelar: (En el load no tengo que hacer nada porque ya me dieron, ya me pusieron desde afuera el modelo)

ACEPTAR: (mismo que antes)

```
1 reference
private void AceptarBoton_Click(object sender, EventArgs e)
{
    //TODO: validar.
    if (!int.TryParse(DocumentoText.Text, out var doc))
    {
        MessageBox.Show("El documento no es válido");
        return;
    }

    //De todas maneras, es un combo así que no hacía falta.
    /*
    if(!Enum.TryParse<TipoTelefono>(TipoCombo.Text, out var tipoTelefono))
    {
        MessageBox.Show("El tipo de telefono no es válido");
        return;
    }
    */

    //etc....
    Persona persona = new()
    {
        Documento = doc,
        Apellido = ApellidoText.Text,
        Nombre = NombreText.Text,
        Telefono = new Telefono
        {
            Tipo = Enum.Parse<TipoTelefono>(TipoCombo.Text),
            CodPais = int.Parse(CodPaisText.Text),
            CodArea = int.Parse(CodAreaText.Text),
            Numero = int.Parse(NumeroText.Text)
        }
    };

    string error = modelo.Modificar(persona);
    if (error != null)
    {
        MessageBox.Show(error);
        return;
```

CANCELAR:

```
1 reference
private void CancelarBoton_Click(object sender, EventArgs e)
{
    this.Close();
}
```

Se hacen las validaciones mínimas (de tipo de dato) y se las paso al modelo para que las agregue a la lista. La única diferencia es que ahora el botón aceptar no recarga la lista. Solo da de alta una nueva persona. Si da un error se lo muestra al usuario y si está todo bien se cierra el formulario.

Cambiando el modelo.Modificar por el modelo.Nuevo

Luego hay que establecer dentro del objeto persona las validaciones de tipo negocio.

Ejemplo de validación:

```
public class Persona
{
    18 references
    public int Documento { get; set; }
    15 references
    public string Nombre { get; set; }
    15 references
    public string Apellido { get; set; }
    18 references
    public Telefono Telefono { get; set; }

    1 reference
    public string Validar()
    {
        //Ej.: verificar que el documento sea > 999_999
        if (Documento < 999_999)
        {
            return "El documento debe tener al menos 7 cifras.";
        }

        //etc... IMPORTANTE: SOLO utilizo MIS propiedades.

        return null;
    }
```

Pero también hay otras validaciones que no las puede hacer el objeto persona, ya que se necesita de más datos. Por ejemplo: 2 personas con el mismo documento. Esas validaciones las hace el modelo, ya que tiene la lista de personas.

```
1 reference
public string Nuevo(Persona nuevaPersona)
{
    string error = nuevaPersona.Validar();
    if (error != null)
    {
        return error;
    }

    //Ej.: no quiero 2 personas con el mismo.
    foreach (var personaExistente in Personas)
    {
        if (personaExistente.Documento == nuevaPersona.Documento)
        {
            return "Ya existe una persona con el documento indicado.";
        }
    }

    Personas.Add(nuevaPersona);

    return null;
}
```

Ya hicimos el formulario "aceptar", ahora vamos con el "editar": Le tenemos que decir al modelo cual es la persona que se está editando, creamos un formulario de edición, le pasamos el modelo y cargamos la lista.
En el formulario principal, el botón editar quedaría así:

```
1 reference
private void EditarBoton_Click(object sender, EventArgs e)
{
    if (PersonasList.SelectedItems.Count == 0)
    {
        MessageBox.Show("Seleccione un ítem de la liste primero.");
        return;
    }

    var personaAEditar = (Persona)PersonasList.SelectedItems[0].Tag;
    modelo.PersonaEnEdicion = personaAEditar;

    var formEdicion = new PersonaEditarForm();
    formEdicion.Modelo = modelo;
    formEdicion.ShowDialog(); //el codigo se detiene...

    //hasta q el formulario se cierra.
    CargarLista();
}
```

Y el PersonaEditarForm_Load quedaría así:

```
1 reference
private void PersonaEditarForm_Load(object sender, EventArgs e)
{
    TipoCombo.Items.Add(TipoTelefono.Casa.ToString());
    TipoCombo.Items.Add(TipoTelefono.Trabajo.ToString());
    TipoCombo.Items.Add(TipoTelefono.Otro.ToString());


    DocumentoText.Text = Modelo.PersonaEnEdicion.Documento.ToString();
    ApellidoText.Text = Modelo.PersonaEnEdicion.Apellido;
    NombreText.Text = Modelo.PersonaEnEdicion.Nombre;

    TipoCombo.SelectedItem = Modelo.PersonaEnEdicion.Telefono.Tipo.ToString();
    CodAreaText.Text = Modelo.PersonaEnEdicion.Telefono.CodArea.ToString();
    CodPaisText.Text = Modelo.PersonaEnEdicion.Telefono.CodPais.ToString();
    NumeroText.Text = Modelo.PersonaEnEdicion.Telefono.Numero.ToString();
}
```

CANCELAR:

```
1 reference
private void CancelarBoton_Click(object sender, EventArgs e)
{
    this.Close();
}
```

ACEPTAR: Igual que antes

```
1 reference
private void AceptarBoton_Click(object sender, EventArgs e)
{
    //TODO: validar.
    if (!int.TryParse(DocumentoText.Text, out var doc))
    {
        MessageBox.Show("El documento no es válido");
        return;
    }

    //De todas maneras, es un combo así que no hacía falta.
    /*
    if(!Enum.TryParse<TipoTelefono>(TipoCombo.Text, out var tipoTelefono))
    {
        MessageBox.Show("El tipo de telefono no es válido");
        return;
    }
    */

    //etc....
    Persona persona = new()
    {
        Documento = doc,
        Apellido = ApellidoText.Text,
        Nombre = NombreText.Text,
        Telefono = new Telefono
        {
            Tipo = Enum.Parse<TipoTelefono>(TipoCombo.Text),
            CodPais = int.Parse(CodPaisText.Text),
            CodArea = int.Parse(CodAreaText.Text),
            Numero = int.Parse(NumeroText.Text)
        }
    };

    string error = Modelo.Modificar(persona);
    if (error != null) //esto es un "FI" => porque es un "IF" al reves.
    {
        MessageBox.Show(error);
        return;
    }

    this.Close();
}
```

Y el Modificar(persona) quedaría asi: (va dentro del PersonasModelo)

```
1 reference
public string Modificar(Persona nuevosDatosPersona)
{
    string error = nuevosDatosPersona.Validar();
    if (error != null)
    {
        return error;
    }

    //Ej.: NO se puede modificar el nro de documento.
    if (PersonaEnEdicion.Documento != nuevosDatosPersona.Documento)
    {
        return "No puede modificar el documento.";
    }

    //Siempre es valido "pisar" los datos sin cambiar el objeto.
    PersonaEnEdicion.ActualizarCon(nuevosDatosPersona);

    return null;
}
```

Notar que editar y nuevo pueden ser muy similares, pero hay que hacerlos a parte para evitar complicaciones. No usar el mismo formulario para ambas operaciones.

Pruebas:
Siempre comienzo probando el camino feliz. No está mal tener errores, lo que está mal es no saber que los tenes. Se puede entregar una versión resaltando los errores que tiene, para luego corregirlos. Pero hay que tener identificados todos los errores.
Las pruebas que cada uno tiene que hacer antes de subir sus cosas (aparte de la prueba del sistema final)son las denominadas smoke test o test rápido. Pienso como programador cuales son las validaciones que puse y las voy rompiendo. Pongo valores que no debería ir, cantidades, negativos, números donde van letras o viceversa, todo.
Ej. El campo “nombre” no puede estar en blanco, en la prueba dió como que se podía, entonces creo una validación:

```
if (string.IsNullOrWhiteSpace(Nombre))
{
    return "El nombre es requerido";
}

return null;
```

Métodos estáticos: no están asignados a un objeto en particular, sino que están asignados a toda la clase. Dependen solamente de los parámetros, no del estado. Ej: IsNullOrWhiteSpace.
Pero por ejemplo el .ToString() no es estático, está asociado a un objeto en particular.

Clase práctica 4 (09/05)
Enfoque en la **estructura de datos** (datos que maneja el sistema). No tiene nada que ver con las pantallas.
El modelo es quien nos divide la estructura de datos de la estructura de las pantallas.
Trabajamos con archivos → ¿Cómo sé que archivos tengo que usar? Primero hay que identificar las clases de datos que maneja el sistema (ej: “personas”). Luego debemos definir qué es una persona. Se define una clase pública, y dentro los tipos de datos que contendrá. Este tipo de clases es de entidad.
No podemos agarrar el modelo Persona y usarlo de archivo → son dos cosas distintas.

1) Armar una carpeta de archivos
2) Armar una estructura de datos de acuerdo a la lógica interna de los datos
3) Crear la clase “Persona”. Usamos public → no trae problemas de restricciones.
4) Definir la persona → agregar las columnas “Documento”, “Nombre”, etc. get set significa que se puede leer y escribir desde afuera.

```
public class Persona
{
    0 references
    public int Documento { get; set; }
    0 references
    public string Nombre { get; set; }
    0 references
    public string Apellido { get; set; }
    0 references
    public DateTime FechaNacimiento { get; set; }
}
```

Modelo: especie lógica entre la estructura de los datos y la estructura de las pantallas

La edad la podemos calcular en un método a partir de la fecha de nacimiento o como una propiedad calculada.

Todos los métodos y propiedades que pongamos dentro de la clase persona tienen que referirse a las propiedades/datos que posee (documento, nombre, etc). Esta clase solo debe hacer las validaciones de los datos que incluye a esas y solamente esas propiedades.

Este modelo es una entidad: una clase que representa los datos grabados en un archivo. No hace referencia a datos que no posee en esta clase.
Para cada clase que yo tenga, voy a tener un archivo de esa clase:

```
public static class ArchivoPersonas
{
    private static List<Persona> personas;

    0 references
    static ArchivoPersonas()
    {
        //se ejecuta UNA sola vez, la primera vez
        //que se hace referencia a la clase ArchivoPersonas.

        if (File.Exists("personas.json"))
        {
            //lee y "transforma" el contenido del archivo personas.json
            //en objetos. Formato = JSON
            var contenido = File.ReadAllText("personas.json");
            personas = JsonConvert.DeserializeObject<List<Persona>>(contenido);
        }
    }
```

Este método debe ser público y estático. Que sea estático quiere decir que no puedo crear instancias de esta clase. Este archivo tiene una lista de personas.

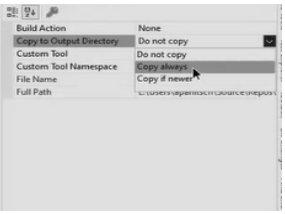
El public static ArchivoPersonas() es un constructor estático- Se ejecuta una sola vez, la primera vez que se hace referencia a la clase ArchivoPersonas.
En el FIle.Exists no se debe poner la ruta del archivo ya que depende de quien lo esté ejecutando, se va a encontrar en una ruta distinta. Solo se pone @"nombrearchivo.json"
Si existe el archivo, lo cargo. Con esto leo el archivo de la base de personas.

Para el caso del TP, los archivos deben ser los mismos que definimos en los DFD.

```
0 references
public static void GrabarDatos()
{
    var contenido = JsonConvert.SerializeObject(personas);
    File.WriteAllText("personas.json", contenido);
}
```

Con esto grabo los archivos (va dentro de la clase ArchivoPersona). Aquí estarán todas las operaciones que modifican datos de una persona.
Con estos dos (cargar y grabar) no necesito más para manejar los archivos.
Deserialize → pasar del archivo a objetos. Serialize → pasar de objetos al archivo.
Para precargar datos (por ej de clientes) creamos un archivo(.json) y ponemos que siempre lo copie al directorio de salida. Siempre lo va a agarrar y lo va a modificar. Se encuentran en la carpeta bin.

```
{
  "Apellido": "Juan",
  "Nombre": "Perez"
},
{
  "Apellido": "Carolingio",
  "Nombre": "Lorena"
}
```



Cuando cargamos los archivos, tocar F4 y elegir siempre copy always para que los mande a la salida y que cada vez que le de F5 el sistema arranque con los mismos datos de siempre

```
0 references
public static void EliminarPersona(int documento)
{
    Persona personaABorrar = null;
    foreach(var persona in personas)
    {
        if (persona.Documento == documento)
        {
            personaABorrar = persona;
            break;
        }
    }

    personas.Remove(personaABorrar);
}
```

Esto no valida, tengo que antes haber creado a la persona para poder borrarla. Sino aparece error

```
public static void AgregarPersona(Persona persona)
{
    personas.Add(persona);
}
```

Los archivos no validan. Las clases archivos solo son responsables de grabar los archivos y hacer operaciones como Agregar, Eliminar o Modificar Persona. Todo esto es dentro de la clase ArchivoPersona.

```
0 references
public static void ModificarPersona(Persona personaAModificar, Persona nuevosDatos)
{
    personaAModificar.Documento = nuevosDatos.Documento;
    //etc...
    //....
}

0 references
public static void ModificarPersona(int documento, Persona nuevosDatos)
{
    personaAModificar.Documento = nuevosDatos.Documento;
    //etc...
    //....
}
```

Hacemos esto para que solo se modifiquen datos acá y sepamos donde está el problema a futuro, si se modificaron mal los datos.
Para todo lo que es de lectura de archivos hacemos una versión de ReadOnly de la lista personas.

```
10
    2 references
11  public static class ArchivoPersonas
12  {
        0 references
13      public static ReadOnlyCollection<Persona> Personas => personas.AsReadOnly();
14
15      private static List<Persona> personas;
16
        0 references
17      static ArchivoPersonas()...
34
        1 reference
35      public static void GrabarDatos()...
40
41      //TODAS LAS OPERACIONES QUE MODIFIQUEN UNA PERSONA.
42
        0 references
43      public static void AgregarPersona(Persona persona)...
```

¿Qué tenemos?

1. Una clase de archivo para cada archivo (Public static class ArchivoPersonas) y mostramos una lista de objetos pública (Personas.asreadOnly())
2. Una clase de entidad para cada archivo (Persona)
    a. Para las modificaciones, debemos tener en cuenta que una persona puede tener varios teléfonos por lo que se vería de esta manera:

```
//No hace referencia a datos que no posee.
9 references
public class Persona
{
    4 references
    public int Documento { get; set; }
    0 references
    public string Nombre { get; set; }
    0 references
    public string Apellido { get; set; }

    0 references
    public List<PersonaTelefono> Telefonos { get; set; } = new();
```

b. Creamos una lista y una clase PersonaTelefono: Hacemos una lista y una clase así y no de Teléfonos porque los teléfonos no existen si no están asignados a una persona.

```
6
7   namespace WinFormsApp1.Archivos;
    1 reference
8   public class PersonaTelefono
9   {
        0 references
10      public string CodPais { get; set; }
        0 references
11      public string CodigoArea { get; set; }
        0 references
12      public string Numero { get; set; }
13  }
14
```

3. Leemos al principio de la ejecución (static ArchivoPersonas()..)
4. Grabamos datos (public static void GrabarDatos())
5. Las modificaciones a esos objetos las hacemos a través de métodos para ver quién me llamó.

```
5   using System.Threading.Tasks;
6
7   namespace WinFormsApp1.Archivos;
    4 references
8   public class Orden
9   {
        0 references
10      public string Codigo { get; set; }
        0 references
11      public DateTime Fecha { get; set; }
        0 references
12      public string Estado { get; set; }
13
14      //algún dato que identifique univocamente al cliente.
        0 references
15      public string ClienteCUIT { get; set; }
16
17      //ESTO ESTA MUUUUUY MAL:
        0 references
18      public Cliente Cliente { get; set; }
19  }
20
```

El archivo de órdenes y el de cliente se van a relacionar con el ClienteCUIT.
Vamos a tener objetos clientes, que tienen los mismos datos pero no son lo mismo. El cliente es uno solo y la orden es una sola.
Si hacemos ese public Cliente Cliente {get; set;} tendríamos datos duplicados. Eso está mal.

Clase práctica 5 (23/05)

```
2 references
public static class ArchivoPersonas
{
    0 references
    public static ReadOnlyCollection<Persona> Personas => personas.AsReadOnly();

    private static List<Persona> personas;

    0 references
    static ArchivoPersonas()...

    1 reference
    public static void GrabarDatos()
    {
        var contenido = JsonConvert.SerializeObject(personas);
        File.WriteAllText(@"Datos\personas.json", contenido);
    }
```

En privado la clase tiene una referencia a la lista que puede ser modificada. En público tiene solo es de lectura (ReadOnly). Permite saber de donde provienen los errores.

Tienen 2 cosas este tipo de clases archivos: un constructor estático que se ejecuta solo una vez y un método para grabar los datos. Los datos se graban una vez terminadas las operaciones.
Los archivos se guardan en formato json.
Estos archivos no son los que se usan cuando el programa se ejecuta. Los datos se copian en una carpeta de salida, y son esas copias las que se usan.
Cada vez que se carga la aplicación va a usar los últimos datos que se corrieron.
La dificultad va a depender de cómo diseñemos las entidades.

Ejemplo entidad/clase “Orden”: tiene propiedades públicas y un método para llevar datos al modelo. Vamos a necesitar de una propiedad que sea una Primary Key y de otras que puedan unirse como Foreign Key de otras entidades. Es como una base de datos no relacional. Trabajamos con documentos y no con tablas.

Si en el DFD tenemos un almacén de órdenes, vamos a tener que codificar una entidad Orden y un archivo de Órdenes. Una entidad, un archivo.
Primero definimos todas las entidades y luego codificamos.
El contenido de los archivos sale de las pantallas. En otros casos debemos tener una lista precargada. Ej: lista de locales de un negocio.

RECUPERATORIO PRACTICA

PARCIAL CAI PRÁCTICA – TEMA 1 – 27/05/2024

Indique verdadero o falso:

F	Cuanta más funcionalidad y operaciones podamos incorporar en un formulario mejor.
V	Es mejor evitar que diferentes formularios se comuniquen entre sí mediante el pasaje de parámetros.
F	Los formularios interactúan directamente con los almacenes de datos.
V	Verificar que un dato no se repita (ej.: no permitir dos clientes con el mismo cuit) es una validación que debe implementarse en el modelo de un formulario.
V	Cada formulario es provisto de datos por un (y sólo un) modelo.
F	Cada modelo corresponde a un almacén de datos.
V	Cada entidad corresponde a un almacén de datos.
F	Como un formulario necesita datos de distintos almacenes, requerirá de varios modelos para acceder a ellos.
F	El formulario debe realizar todas las validaciones necesarias para asegurar la corrección de los datos ingresados por el usuario. Los modelos sólo se encargan de proveer los datos iniciales.
F	Cada almacén está relacionado a varias entidades.

1. F
2. V
3. F → estan los modelos en el medio
4. V: el formulario chequea la validez del dato pero puede que este repetido por ejemplo
5. V
6. F
7. V
8. F → se contradice con la 5
9. F → se contradice con la 4
10. F → se contradice con la 7

Seleccione la estructura más adecuada para representar una factura (la sintaxis de c# es válida en todas las opciones, no busque errores):

```
public class Factura
{
    public string CUIT { get; set; }
    public string RazonSocial { get; set; }
    public decimal Neto { get; set; }
    public decimal IVA { get; set; }
    public decimal Total { get; set; }
    public FacturaDetalle[] Detalle { get; set; }
}

public class FacturaDetalle
{
    public string Descripcion { get; set; }
    public decimal Neto { get; set; }
    public decimal IVA { get; set; }
    public decimal Total { get; set; }
}
```

```
public class Factura
{
    public string CUIT { get; set; }
    public string RazonSocial { get; set; }
    public decimal Neto { get; set; }
    public decimal IVA { get; set; }
    public decimal Total { get; set; }

    //Detalle:
    public List<string> Descripcion { get; set; }
    public List<decimal> Neto { get; set; }
    public List<decimal> IVA { get; set; }
    public List<decimal> Total { get; set; }
}
```

```
public class Factura
{
    public string CUIT { get; set; }
    public string RazonSocial { get; set; }
    public decimal Neto => Detalle.Sum(d => d.Neto);
    public decimal IVA => Detalle.Sum(d => d.IVA);
    public decimal Total => Detalle.Sum(d => d.Total);
    public List<FacturaDetalle> Detalle { get; } = new();
}

public class FacturaDetalle
{
    public string Descripcion { get; set; }
    public decimal Neto { get; set; }
    public decimal IVA { get; set; }
    public decimal Total => Neto + IVA;
}
```

```
public class Factura
{
    public string CUIT { get; set; }
    public string RazonSocial { get; set; }
    public decimal Neto => Detalle.Sum(d => d.Neto);
    public decimal IVA => Detalle.Sum(d => d.IVA);
    public decimal Total => Detalle.Sum(d => d.Total);
    public List<FacturaDetalle> Detalle { get; set; }
}

public class FacturaDetalle
{
    public string Descripcion { get; set; }
    public decimal Neto { get; set; }
    public decimal IVA { get; set; }
    public decimal Total { get; set; }
}
```

ABAJO A LA IZQUIERDA

el 1 tiene un array de Detalle. ARRAY NO VA
Si tiene New SI
La última no está mal.

Dada la linea de código:

```
var variable1 = int.TryParse(Console.ReadLine(), out var variable2);
```

Indique V o F:

a) Variable1 es de tipo bool V
b) Si el usuario ingresa "hola" se produce un error F
c) Si el usuario ingresa "44" variable1 será false F
d) Variable2 es de tipo int V
e) Si el usuario ingresa "44" variable2 será igual a 44. V

a) VERDADERO
b) FALSO → el try parse nunca da error
c) FALSO
d) VERDADERO
e) VERDADERO

Marque con una cruz todos los ValueType

X	Double
X	DateTimeOffset
X	DateTime
	Queue
	List
	Persona
	File
X	Int
X	String
X	Long
	Dictionary
X	Single

¿Qué significa la siguiente sintaxis?

Auto obj = new Auto();

X	Estamos asignando la clase Auto.
SI	Estamos instanciando un objeto de tipo Auto.
X	Estamos creando una nueva clase "Auto".
SI	Estamos asignando un objeto auto a una variable llamada obj.
NO	Ninguna es correcta.
NO	Todas son correctas

ValueType: Todos los que representan valor (string si)

Seleccione el tipo de datos más adecuado para representar...

	int	decimal	bool	DateTime	TimeSpan	string
El horario de salida de un vuelo				OK	OK	
El número de CUIT						SI
Edad de una persona	X					
El estado encendido/apagado de un interruptor de luz			X			
El precio de un producto		X				
La fecha y hora de una cita				OK	OK	
Nombre y apellido						OK
Tiempo de un viaje					OK	
Si se contactó a un cliente o no			X			
Peso en kg.		X				
Stock en unidades de un producto	X					
El estado de un semáforo rojo/amarillo/verde.	X					
Cantidad de páginas web visitadas	X					
Total de bolsas de cemento vendidas	X					
Litros de agua en un depósito		X				
Porcentaje de IVA		X				
Total Neto de una factura		X				
Tiempo de espera para un turno					X	
Horas trabajadas en el mes	X				X	
El contenido de un archivo (no sabemos de qué)						X

PARCIAL CAI PRÁCTICA – TEMA 2 – 27/05/2024

Indique verdadero o falso:

V	En general, un formulario debe responder a una (y solo una) operación de negocio.
F	Es mejor que los formularios actúen de manera coordinada, comunicándose a través del pasaje de parámetros.
V	Los formularios no interactúan directamente con los almacenes de datos.
F	Es más eficiente si utilizamos el mismo modelo en varios formularios similares.
F	Unificamos el manejo de todos los almacenes de datos en un solo objeto.
F	Una entidad corresponde a varios almacenes de datos
V	Como un formulario necesita datos de distintas entidades, su modelo deberá consultar y relacionar los datos de todos los almacenes necesarios para proveerlos de manera unificada.
V	Verificar que el usuario haya ingresado una fecha sea válida es una típica validación que debe implementarse en un formulario.
F	Para crear un prototipo, primero debemos ponernos de acuerdo en la estructura de datos del sistema.
V	Cada almacen está relacionado con una y sólo una entidad.

```
XXXpublic class Factura
{
    public string CUIT { get; set; }
    public string RazonSocial { get; set; }
    public decimal Neto { get; set; }
    public decimal IVA { get; set; }
    public decimal Total { get; set; }
    public FacturaDetalle[] Detalle { get; set; }
}

public class FacturaDetalle
{
    public string Descripcion { get; set; }
    public decimal Neto { get; set; }
    public decimal IVA { get; set; }
    public decimal Total { get; set; }
}
```

```
public class Factura
{
    public string CUIT { get; set; }
    public string RazonSocial { get; set; }
    public decimal Neto => Detalle.Sum(d => d.Neto);
    public decimal IVA => Detalle.Sum(d => d.IVA);
    public decimal Total => Detalle.Sum(d => d.Total);
    public List<FacturaDetalle> Detalle { get; } = new();
}

public class FacturaDetalle
{
    public string Descripcion { get; set; }
    public decimal Neto { get; set; }
    public decimal IVA { get; set; }
    public decimal Total => Neto + IVA;
}
```

```
XXX public class Factura
{
    public string CUIT { get; set; }
    public string RazonSocial { get; set; }
    public decimal Neto { get; set; }
    public decimal IVA { get; set; }
    public decimal Total { get; set; }

    //Detalle
    public Li: <string> Descripcion { get; set; }
    public Li: <decimal> Neto { get; set; }
    public Li: <decimal> IVA { get; set; }
    public Li: <decimal> Total { get; set; }
```

```
XXXXpublic class Factura
{
    public string CUIT { get; set; }
    public string RazonSocial { get; set; }
    public decimal Neto => Detalle.Sum(d => d.Neto);
    public decimal IVA => Detalle.Sum(d => d.IVA);
    public decimal Total => Detalle.Sum(d => d.Total);
    public List<FacturaDetalle> Detalle { get; set; }
}

public class FacturaDetalle
{
    public string Descripcion { get; set; }
    public decimal Neto { get; set; }
    public decimal IVA { get; set; }
    public decimal Total { get; set; }
}
```

ARRIBA A LA DERECHA

Dada la linea de código:

```
var variable1 = decimal.TryParse(Console.ReadLine(), out var variable2);
```

Indique V o F:

a) variable1 es de tipo bool SI
b) Si el usuario ingresa “44” variable1 será false FALSE
c) variable2 es de tipo int NO
d) Si el usuario ingresa “hola” se produce un error NO
e) Si el usuario ingresa "33.5" variable2 será igual a 33.5 SI

Marque con una cruz todos los ValueType

X	Bool
	Vuelo
	File
X	DateTime
X	Single
	List
X	Double
	Persona
X	Long
	Object
X	String
X	DateTimeOffset

¿Qué significa la siguiente sintaxis?

Auto obj = new Auto();

F	Estamos instanciando un objeto de tipo base.
F	Estamos duplicando un objeto.
V	Estamos creando un nuevo objeto.
	Estamos instanciando una nueva clase.
F	Ninguna es correcta.
F	Todas son correctas

Seleccione el tipo de datos más adecuado para representar...

	int	decimal	bool	DateTime	TimeSpan	string
El horario de salida de un vuelo				X	X	
El número de CUIT						X
Edad de una persona	X					
El estado encendido/apagado de un interruptor de luz			X			
El precio de un producto		X				
La fecha y hora de una cita				X		
Nombre y apellido						X
Tiempo de un viaje					X	
Si se contactó a un cliente o no			X			
Peso en kg.		X				
Stock en unidades de un producto	X					
El estado de un semáforo rojo/amarillo/verde.	X					
Cantidad de páginas web visitadas	X					
Total de bolsas de cemento vendidas	X					
Litros de agua en un depósito		X				
Porcentaje de IVA		X				
Total Neto de una factura		X				
Tiempo de espera para un turno					X	
Horas trabajadas en el mes	X				X	
El contenido de un archivo (no sabemos de qué)						X

CAI – RECUPERATORIO PRACTICA

1) Dada la línea de código:

var variable1 = int.TryParse(Console.ReadLine(), out var variable2);

indique las afirmaciones incorrectas (VERDE CORRECTO, ROJO INCORRECTO)

a) variable1 es de tipo bool.

b) Si el usuario ingresa "hola" se produce un error.

c) Si el usuario ingresa "44" variable1 será false.

d) Si el usuario ingresa "44" variable2 será igual a 44.

e) Si el usuario ingresa "44" variable1 es igual a 44.

f) variable1 es de tipo int

g) Si el usuario ingresa "hola" variable1 será false.

2) Indique la definición de clase que se ajusta al siguiente fragmento de código:

var a = new Alumno(999774, "Juan", "Perez");

Opción 1:

```
class Alumno
{
    public string Nombre { get; set; }
    public string Apellido { get; set; }
    public long Registro { get; set; }
}
```

Opcion 2:

```
class Alumno
{
    public static Alumno Crear(long registro, string nombre, string apellido)
    {
        return new Alumno(registro, nombre, apellido);
    }

    public string Nombre { get; set; }
    public string Apellido { get; set; }
    public long Registro { get; set; }
}
```

Opcion 3:

```
class Alumno
{
    public static Alumno Crear(long registro, string nombre, string apellido)
    {
        return new Alumno(registro, nombre, apellido);
    }

    public Alumno(long registro, string nombre, string apellido)
    {
        Registro = registro;
        Nombre = nombre;
        Apellido = apellido;
    }

    public string Nombre { get;  }
    public string Apellido { get;  }
    public long Registro { get; }
```

a) Opción 1
b) Opción 2
c) Opción 3 → tiene tanto un constructor que acepta tres parámetros (registro, nombre y apellido) como un método estático `Crear` que devuelve una nueva instancia de `Alumno`. Esta opción define la clase Alumno con un constructor(public Alumno) que inicializa las propiedades Registro, Nombre y Apellido, además de un método estático Crear para crear instancias de Alumno.

3) ¿Cuál/es de los siguientes tipos de datos es numérico? (VERDE SI, ROJO NO)

a) bool
b) int
c) double
d) decimal
e) Todos son numéricos

4) bool resultado = ((true && true) && (false == false)) || (1 == 1) || (5 * 5 == 24);
La variable resultado será...

a) true (si hay un true ya es true)
b) false
c) null

5) Indique la definición de clase que se ajusta al siguiente fragmento de código:
var factura = Archivo.ObtenerFactura("A-8005-00004455")

Opcion 1:

```
class Archivo
{
    private static Factura ObtenerFactura(string numero)
    {
        /* se omite el cuerpo del método */
    }
}
```

Opcion 2:

```
class Archivo
{
    public static Factura ObtenerFactura(string numero)
    {
        /* se omite el cuerpo del método */
    }
}
```

Opcion 3:

```
static class Archivo
{
    internal Factura ObtenerFactura(string numero)
    {
        /* se omite el cuerpo del método */
    }
} *
```

a) Opción 1
b) Opción 2 → Si el método es private sólo sería accesible dentro de la clase Archivo y si es static no permitiría la creación de instancias de la clase Archivo
c) Opción 3

6) Las clases se componen de atributos, sus métodos y los mensajes identificados entre clases. En forma genérica, podemos decir que existen clases de entidad, de interfaz / límites, y de control o coordinación. *

a) Verdadero
b) Falso

Las clases entidad son las que sirven para modelar los datos, por ejemplo crear una clase Persona que agrupe todos sus atributos. Las clases de control son más que se encargan de realizar lo que el usuario desea. Las clases de interfaz son las que se encargan de las vistas, es decir lo que el usuario ve.

7) Qué significa la siguiente sintaxis?
Auto obj = new Auto(); *

a) Estamos declarando una abstracción.
b) Estamos instanciando un objeto de tipo base.
c) Estamos instanciando un objeto de tipo Auto.
d) Estamos creando una nueva clase.
e) Ninguna es correcta.

8) Seleccione la opción que completa la instrucción for produciendo la secuencia de números 0, 2, 4, 6, ... 18 (todos los pares entre 0 y 18 inclusive).

```
for( ... )
{
    Console.WriteLine(i);
}
```

a) int i = 0; i <= 20; i =+ 2
b) int i = 0; i < 20; i++++

c) int i = 0; i < 20; i += 2 → Este bucle comienza en 0 y se incrementa de 2 en 2, asegurándose de que `i` sea menor o igual a 18
d) int i = -2; i <= 20; i += 2

9) ¿Cual es la clase más apropiada para almacenar una fecha? DateTime
10) ¿Cual es la clase más apropiada para almacenar una fecha (sin zona horaria)? DateTime
11) ¿Y si es relevante la zona horaria (ej: 15.30 GMT -3)? DateTimeOffset o TimeZone

//almacena un punto en el tiempo absoluto con información de zona horaria precisa, mientras que ZoneTime sólo almacena información sobre una zona horaria específica

12) ¿Con qué clase representaría ud. el precio de un producto? (VERDE SI, ROJO NO)

a) Precio
b) long
c) Producto
d) decimal
e) double
f) single
g) int

13) Seleccione el código que debe incluirse en la línea 3 para que el programa funcione correctamente:

```
1       using(var r = new
StreamReader(File.OpenRead(@"C:\CAI\Alumnos.txt")))
2       {
3          /*Seleccione la línea que completa el código*/
4          {
5             var linea = r.ReadLine();
6             var a = new Alumno(linea);
7             Console.WriteLine($"{a.Registro} - {a.Apellido}, {a.Nombre}");
8          }
9       } *
```

a) while(r)
b) while(r.ReadLine() !== null)
c) while(r.EndOfStream)
d) while(!r.EndOfStream)
e) while(r.EndOfStream !== null)

14) ¿Qué instrucción es más apropiada para declarar una variable que almacene exactamente 15 números enteros?

a) var lista = new int[15];
b) var lista = new int[14];
c) var lista = new List<int>(15);
d) var lista = new List<int>(14);
e) var lista = new List<int>();

15) ¿Qué instrucción es más apropiada para declarar una variable que almacene exactamente 15 números enteros, dado que nunca se agregara o quitara un numero de este conjunto?

a) var lista = new int[15];
b) var lista = new int[14];
c) var lista = new List<int>(15);
d) var lista = new List<int>(14);
e) var lista = new List<int>();

16) ¿Cuántos objetos Alumno se mantienen en memoria?

```
var a = new Alumno();
var b = a;
var a = new Alumno();
var c = b;
var b = null; *
```

a) 1
b) 2
c) 3
d) 4
e) 5

17) Indique las líneas de código que son incorrecta para completar la instrucción foreach para obtener las facturas del mes pasado. Las opciones indican el código a considerar en el espacio marcado con "..." :

```
foreach( .... )
{
   Console.WriteLine(factura);
} *
```

a) var factura in Archivo.ObtenerFacturas(DateTime.Today.AddDays(-DateTime.Today.Day))
b) var factura in Archivo.ObtenerFacturas(DateTime.Today.AddMonth(-1))
c) var factura in Archivo.ObtenerFacturas(DateTime.Today.AddDays(-DateTime.Today.Day + 1))
d) var factura in Archivo.ObtenerFacturas(DateTime.FirstDayOfMonth)))

18) El metodo ObtenerFacturas de la clase archivo devuelve las facturas posteriores a la fecha y hora que se le pasa como parámetro. Complete la instrucción foreach para obtener las facturas a partir del primero del mes actual a las 0 horas. Las opciones indican el código a incluir en el espacio marcado con "...":

```
foreach( .... )
{
   Console.WriteLine(factura);
} *
```

a) var factura in Archivo.ObtenerFacturas(DateTime.Today.AddDays(-DateTime.Today.Day))
b) var factura in Archivo.ObtenerFacturas(DateTime.Today.AddMonth(-1))
c) var factura in Archivo.ObtenerFacturas(DateTime.Today.AddDays(-DateTime.Today.Day + 1))
d) var factura in Archivo.ObtenerFacturas(DateTime.FirstDayOfMonth)))

19) ¿Qué significa la palabra reservada **void** que se emplea en un método? *

a) Se refiere a la capacidad de ese método de procesar.
b) Se refiere a que va a devolver null.
c) Se refiere a que no tiene valor de retorno.
d) Ayuda a garantizar la interoperabilidad de la clase en términos de polimorfismo.
e) Ninguna es correcta.

20) Marque las afirmaciones verdaderas teniendo en cuenta el siguiente fragmento de código:

```
class Program
{
  static void Main(string[] args)
  {
     Console.WriteLine("Ingrese un numero decimal negativo:");
     var ingreso = Console.ReadLine();
     var esCorrecto = decimal.TryParse(ingreso, out var resultado);
  }
} *
```

a) Puede producirse un error en tiempo de ejecución, en caso de que ingreso sea null. *decimal.TryParse maneja el valor null devolviendo false y asignando 0 a resultado sin lanzar una excepción.*
b) WriteLine es un método de instancia. *Es un método estático de la clase Console.*
c) Main es un método estático de la clase Program.
d) WriteLine es un método estático de la clase Console.
e) Program es una instancia de la clase Console. *Program es una clase independiente, no tiene relación directa de instancia con la clase Console*
f) esCorrecto es una variable de tipo bool y resultado de tipo decimal.
g) WriteLine es un método de instancia en una clase estática. *WriteLine es un método estático en la clase estática Console. No es un método de instancia.*
h) decimal.TryParse devuelve un objeto de tipo string. *Devuelve un valor de tipo bool, que indica si la conversión fue exitosa o no*
i) decimal.TryParse devuelve un objeto de tipo decimal. *Devuelve un valor de tipo bool. El valor parseado se asigna a la variable out (en este caso, resultado), que es de tipo decimal.*
j) Ingreso es una variable tipo string.
k) ingreso es una variable de tipo char[] (array de caracteres).
l) resultado es una variable de tipo double. *Es de tipo decimal, ya que es el tipo especificado en decimal.TryParse.*
m) Console.WriteLine es una propiedad sin valor de retorno. *Es un método estático, no una propiedad*

21) Responda V/F teniendo en cuenta el siguiente fragmento de código:

```
class Program
{
  static void Main(string[] args)
  {
    Console.WriteLine("Ingrese un numero decimal negativo:");
    var ingreso = Console.ReadLine();
    bool esCorrecto = decimal.TryParse(ingreso, out var resultado);
  }
}
```

a) Console es una instancia de la clase Program.
b) WriteLine es un método de instancia en una clase estática.
c) decimal.TryParse devuelve un objeto de tipo string.
d) WriteLine es un método estático de la clase Console.
e) ingreso es una variable de tipo string.
f) resultado es una variable de tipo double.

22) ¿En qué tipo de clase de colección almacenaría ud. la lista de pacientes que deben ser atendidos por el médico, teniendo en cuenta que debe respetarse el orden de llegada?

a) Paciente[]
b) Dictionary<long, Paciente>
c) Queue<Paciente>
d) List<Paciente>
e) Stack<Paciente>
f) SortedSet<Paciente>

23) ¿Y si los pacientes fuesen llamados arbitrariamente por el médico, por número de documento? *

a) Paciente[]
b) Dictionary<long, Paciente>
c) Queue<Paciente>
d) List<Paciente>
e) Stack<Paciente>
f) SortedSet<Paciente>

24) Dada las variables horaInicio y horaFin, seleccione el código que determina la diferencia en segundos entre las dos//Suponiendo que las variables horaInicio y horaFin referencian a objetos de tipo Timespan, seleccione el código que determina la diferencia en segundo entre las dos.

a) var segundos = horaFin - horaInicio;
b) var segundos = (horaFin - horaInicio).TotalSeconds;
c) var segundos = new TimeSpan(horaFin - horaInicio).TotalSeconds;
d) var segundos = (horaFin - horaInicio).Seconds;
e) var segundos = horaFin.TotalSeconds - horaInicio.TotalSeconds;

25) Qué instrucción es más apropiada para declarar una variable que almacene una cantidad arbitraria de números enteros? *
a) var lista = new int[];
b) var lista = new int[] { 2, 4, 5, ... };
c) var lista = new List<int>();
d) Ninguna de las anteriores

26) Seleccione todos los que son ValueType * (VERDE SON, ROJO NO SON)
a) bool
b) Dictionary
c) DateTime
d) DateTimeOffset
e) List
f) Queue
g) File
h) long
i) string
j) Int64
k) Double
l) single
m) Object
n) Persona
o) Vuelo

27) Marque las afirmaciones correctas. (VERDE CORRECTAS, ROJA INCORRECTAS) a ver como resolves este...
a) Una clase es la definición a partir de la cual se construye una instancia de objeto.
b) El término clase refiere a la definición a partir de la cual son construidos los objetos.
c) El valor null es el valor por defecto de cualquier variable. // Todas las variables tienen el valor null por defecto
d) El .Net Framework es un lenguaje de programación orientado a objetos.
e) Una clase estática es aquella de la que sólo existe una única instancia
f) Un objeto de tipo Dictionary representa una colección de pares clave/valor.
g) Una variable es siempre una referencia a un objeto.
h) Un método estático debe invocarse utilizando el nombre de la clase que lo contiene.
i) Un método no estático debe invocarse en una instancia de objeto en particular.
j) Un método estático pertenece a una instancia de clase específica.
k) Es obligatorio utilizar “var” en aquellos casos donde indicar un tipo específico es redundante.
l) El .Net Framework es una colección de librerías que contienen clases (de uso general).
m) Un objeto es la instancia de la clase que lo define
n) Si una clase es estática, el sistema sólo permite crear y asignar a variables una única instancia de ella.
En C#, si una clase es estática, no se permite crear ninguna instancia de ella en absoluto.
o) Una variable con valor distinto de null contiene una referencia a un objeto. // Una variable con valor distinto de null puede contener un valor o una referencia a un objeto.
p) Una variable con valor null no tiene un tipo determinado.

q) Una variable (con valor distinto de null) contiene una referencia a un objeto de la clase definida para la variable
r) Un método de instancia debe invocarse a partir de una referencia a un objeto en particular.
s) Una variable definida con "var" no tiene un tipo definido.
Sí tiene un tipo definido, pero este tipo se infiere automáticamente del tipo de la expresión a la que se asigna en el momento de la inicialización.
t) Una variable de tipo int contiene un número entero dentro de un rango determinado
u) Pueden crearse varios objetos a partir de la misma definición de clase.
v) Un objeto de tipo Stack representa una colección de pares clave/valor.
w) La clase List<T> representa una lista de elementos, donde T es el tamaño o capacidad de la misma.
T es el tipo de dato que va a tener la lista

28) ¿Cuantos objetos Alumno se han creado?

```
var a = new Alumno();
var b = a;
var c = b;
var d = new Alumno();
var c = d; *
```

a) 1
b) 2
c) 3
d) 4
e) 5

29) ¿Con qué clase representaría la duración de un plazo fijo?

a) DateTime[2]
b) DateTimeOffset[2]
c) DateTime
d) TimeSpan → Representa un intervalo de tiempo
e) DateTimeOffset
f) TimeSpan[2]

33) Indique la declaración más apropiada para cada caso:

a) Almacenar una serie de números enteros.
int[] numeros;
List<int> numeros;
Dictionary numeros;
int numeros;
List<decimal> numeros;

b) Representar una pila de trabajo (primero entrado – ultimo salido).
List<Item> trabajo;
Stack<Item> trabajo;
var trabajo = new Stack();
Dictionary<Item,
int> trabajo;

c) Almacenar, para una cantidad de alumnos, su nombre y número de registro.
Dictionary<int, string> alumnos;
List<int,string> alumnos;
Dictionary<string, int> alumnos;
KeyValue<int,string>[] alumnos;

d) Exactamente 10 fechas:
var fechas = new DateTime[10];

DateTime[9] fechas;
DateTime[10] fechas = new DateTime[];
List<DateTime> fechas;

e) Representar la cola de clientes en un mostrador:
List<Cliente> clientes;
Queue<Cliente> clientes;
var clientes = new Cliente[]{};
HashSet<Cliente> clientes;

f) El intervalo entre dos fechas:
DateTime[] fechas;
decimal segundos;
TimeStamp segundos;
TimeSpan intervalo;

g) Almacenar una cantidad indefinida de precios.
double[] precios;
List precios;
decimal precios = new decimal[];
List<decimal> precios;
Var precios;

h) Almacenar fechas de caducidad por producto
Dictionary<string, DateTime> fechas;
Dictionary<DateTime,string> fechas;
(string, DateTime) fechas;
(DateTime, string) fechas;

i) Exactamente 15 valores booleanos
var valores = new bool[14];
var[15] valores = new bool [];
Dictionary<bool> valores;
var valores = new BitArray (15;)

j) Representar una fecha
var fecha = new TimeSpan();
var fecha = DateTime (2005-05-12);
DateTime fecha;
DateTime fecha = #2012-11-5#

k) Almacenar una serie de nombre:
string[] nombres;
List nombres;
string nombres = new string [];
List<string> nombres;
List<char> nombres;

l) Almacenar precios por código de producto:
List <decimal, string>
(string, decimal)[][] precios;
Dictionary<string, decimal> precios;
Dictionary<decimal,string> precios

m) Exactamente 15 valores enteros:
var valores = new int[14];
var valores = new int[15];
int[15] valores;
List<int> fechas;

32) Escriba un ejemplo de código válido y sin errores de ejecución para cada caso, sin superar el espacio asignado.

a) Creación de un objeto:
b) Invocación de un método estático:
c) Invocación de un método de instancia:
d) Asignar a una variable el ingreso del usuario en la consola:

e) Asignar a una variable de tipo DateTime una fecha ingresada por el usuario (en caso de que el ingreso sea una fecha válida – sin reingreso en caso de error).

34) Codifique aplicaciones de consola para:

1. Calcular el promedio de una serie de números ingresados por el usuario.
2. Solicitar el ingreso de una serie de cadenas y mostrarlas en orden inverso.
3. Solicitar el ingreso de una serie de códigos de producto (exactamente 4 caracteres) y precios hasta que el usuario ingrese “XXXX” como código. A continuación, un ciclo que muestre el precio de un producto ingresado por el usuario hasta que éste ingrese “XXXX”.

Utilice las validaciones que considere necesarias para validar el ingreso del usuario (permitiendo reingreso en caso de error) y evitar errores de ejecución.

Marque V o F según corresponda

1. Una clase es la definición a partir de la cual se construye una instancia de objeto. V
2. El valor null es el valor por defecto de cualquier variable. F
4. Un objeto de tipo Dictionary representa una colección de pares clave/valores. V
5. Una variable es siempre una referencia a un objeto. F
6. Un método estático debe invocarse utilizando el nombre de la clase que lo contiene. V
7. Un método estático pertenece a una instancia de clase específica. F
8. Una variable con valor null no tiene un tipo determinado. V
9. Una variable (con valor distinto de null) contiene una referencia a un objeto de la clase definida para la variable V
10. Es obligatorio usar “Var” en casos donde indicar un tipo específico es redundante. F
11. El término clase refiere a la definición a partir de la cual son construidos los objetos. V
12. Un método no estático debe invocarse en una instancia de objeto en particular. V
13. Una variable de tipo int contiene un número entero dentro de un rango determinado V
14. El .Net Framework es una librería de clases de uso general. V
15. Un objeto es la instancia de la clase que lo define V
16. El valor null se convierte al valor predeterminado de cada tipo específico. F
17. Si una clase es estática, el sistema sólo permite crear y asignar a variables una única instancia de ella. V
18. Una variable con valor distinto de null contiene una referencia a un objeto. V
19. Un método de instancia debe invocarse a partir de una referencia a un objeto en particular. V
20. Una variable definida con “var” no tiene un tipo definido. F
21. Pueden crearse varios objetos a partir de la misma definición de clase. V
22. Todas las variables tienen el valor null por defecto. F
23. Un objeto de tipo Stack representa una colección de pares clave/valor. F
24. Una clase estática es aquella de la que sólo puede asignarse a variables una única instancia. V
25. La clase List<T> representa una lista de elementos, donde T es el tamaño o capacidad de la misma. (T es el tipo de dato que va a tener la lista)- F
26. Un método estático debe invocarse utilizando el nombre de la clase que lo contiene. V
27. Es obligatorio utilizar “var” en aquellos casos donde indicar un tipo específico es redundante. F
29. WriteLine es un método de instancia F
30. MAIN es un método estático de la clase Program. V
31. WriteLine es un método estático de la clase Console. V
32. Program es una instancia de la clase Console. V
33. WriteLine es un método de instancia en una clase estática. F
34. decimal.TryParse devuelve un objeto de tipo string. F
35. decimal.TryParse devuelve un objeto de tipo decimal. F
36. WriteLine es un método estático de la clase Console. V
37. Ingreso es una variable tipo string. V
38. ingreso es una variable de tipo char[] F
39. Resultado es una variable de tipo double. F
40. Console es una instancia de la clase Program. F

41. El resultado es una variable de tipo decimal. V
42. Console.WriteLine es una propiedad sin valor de retorno F
43. ReadLine es un método de instancia F
44. TryParse es un método de la clase decimal - V
45. Si ingreso pepe se produce un error y esCorrecto = false F
46.¿Qué clase es más apropiada para almacenar exactamente 15 números enteros?
int[] ejemplo = new int[15]
47.¿Y una cantidad arbitraria de números enteros?
List<int> listaNumerosEnteros = new List<int>()